

AGH

AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

WYDZIAŁ INFORMATYKI, ELEKTRONIKI I TELEKOMUNIKACJI

INSTYTUT *Elektroniki*

PRACA DYPLOMOWA INŻYNIERSKA

Elektroniczny krokomierz z archiwizacją danych

Electronic pedometer with data archiving

Autor:

Kierunek studiów:

Opiekun pracy:

Maciej Filipiak

Elektronika i Telekomunikacja

prof. dr hab. inż. Andrzej Kos

Kraków, 2025/2026

Spis treści

SPIS STOSOWANYCH SYMBOLI I SKRÓTÓW UŁOŻONY ALFABETYCZNIE.....	3
WSTĘP	4
ROZDZIAŁ 1 CZĘŚĆ OPISOWA.....	6
1.1 ANALIZA ROZWIĄZAŃ RYNKOWYCH	6
1.2 BUDOWA I ZASADA DZIAŁANIA AKCELEROMETRÓW TYPU MEMS	7
1.3 CHARAKTERYSTYKA SYGNAŁU PRZYSPIESZENIA PODCZAS CHODU CZŁOWIEKA	8
1.4 INTERFEJSY KOMUNIKACYJNE W SYSTEMACH IoT	10
1.4.1 Magistrala I ² C	11
1.4.2 Wi-Fi w systemach wbudowanych.....	11
ROZDZIAŁ 2 PROJEKT I REALIZACJA URZĄDZENIA	14
2.1 ZAŁOŻENIA FUNKCJONALNE I TECHNICZNE	14
2.2 DOBÓR KOMPONENTÓW SPRZĘTOWYCH	14
2.2.1 Jednostka centralna ESP32 Dev Kit V1	15
2.2.2 Moduł pomiarowy MPU 6050	17
2.2.3 Wyświetlacz OLED	21
2.2.4 Płytki stykowe	23
2.2.5 Źródło zasilania mobilnego	24
2.3 MONTAŻ URZĄDZENIA	25
2.4 ALGORYTM DZIAŁANIA OPROGRAMOWANIA	26
2.5 IMPLEMENTACJA OPROGRAMOWANIA	28
2.5.1 Środowisko programistyczne i wykorzystane biblioteki.....	28
2.5.2 Opracowanie algorytmu detekcji kroków.....	29
2.5.3 Inicjalizacja systemów łączności i interfejsu wizualnego	31
2.5.4 System archiwizacji danych i serwer WWW	32
2.6 PROJEKT OBUDOWY	34
2.6.1 Wydruk prototypu obudowy.....	35
2.7 TESTY POPRAWNOŚCI DZIAŁANIA	38
2.7.1 Metodyka badań.....	38
2.7.2 Zestawienie wyników.....	38
2.7.3 Analiza porównawcza	38
2.7.4 Wpływ charakteru trasy na błąd pomiarowy	39
PODSUMOWANIE I WNIOSKI	40
BIBLIOGRAFIA	41
SPIS RYSUNKÓW	43

Spis stosowanych symboli i skrótów ułożony alfabetycznie

ADC – Analog-to-Digital Converter

DMP – Digital Motion Processor

g – stała przyspieszenia ziemskiego w przybliżeniu $9,81 \text{ [m/s}^2\text{]}$

GPIO – General Purpose Input/Output

HTML – Hypertext Markup Language

HTTP – Hypertext Transfer Protocol

I²C – (ang. Inter-Integrated Circuit) Szeregowa, dwukierunkowa magistrala komunikacyjna

IoT – Internet of Things

JSON – JavaScript Object Notation

LDO – Low-Dropout Regulator

MEMS – Micro-Electro-Mechanical Systems

NVS – Non-Volatile Storage

OLED – Organic Light-Emitting Diode

SRAM – Static Random Access Memory

WLAN – Wireless Local Area Network

WPA – Wi-Fi Protected Access

WWW – World Wide Web

Wstęp

Od bardzo wielu lat w dziedzinie technologii mobilnych i systemów wbudowanych trwa dynamiczny i nieustanny rozwój. W efekcie w naszym życiu coraz poważniejszą rolę odgrywają ogólnodostępne i kompaktowe urządzenia pomiarowe różnego typu. Urządzenia te sprawdzają się doskonale w kontroli procesów przemysłowych, monitorowaniu parametrów środowiskowych a także monitorowaniu aktywności fizycznej i stanu zdrowia użytkowników. Przykładami takich rozwiązań są elektroniczne krokomierze, których głównym zadaniem jest analiza aktywności ruchowej użytkownika, w tym przede wszystkim zliczanie kroków przebytych podczas trasy i na tej podstawie przestawienie tempa oraz intensywności aktywności. Dzisiejsze krokomierze bardzo często stanowią tylko element bardziej rozbudowanych systemów obejmujących aplikacje mobilne. Połączenie technologii komunikacyjnych i urządzeń pomiarowych daje możliwość bieżącego dostępu do danych w czasie rzeczywistym, a także przy zastosowaniu archiwizacji do danych z wcześniejszych sesji treningowych, co może być pomocne w celu monitorowania swoich postępów i ustalania nowych rekordów. Urządzenia te łączą wiele dziedzin i zagadnień z zakresu elektroniki, bezprzewodowej transmisji danych, systemów wbudowanych i programowania mikrokontrolerów. Złożoność tych zagadnień stała się fundamentem do zaprojektowania i budowy własnego systemu pomiarowego [1].

W niniejszej pracy przedstawiono projekt od strony teoretycznej, jak i praktycznej. W rozdziale podstawy teoretyczne znajdują się kompleksowe wyjaśnienie podstawowych zagadnień potrzebnych do realizacji. Natomiast w drugiej, praktycznej części skupiono się przede wszystkim na samej realizacji projektu krokomierza, na co składa się szczegółowa analiza doboru i integracji odpowiednich komponentów elektronicznych, montaż urządzenia, opracowanie i zaimplementowanie algorytmu sterującego działaniem krokomierz oraz wykonanie obudowy prototypu.

Cel pracy

Celem mojej pracy inżynierskiej było zaprojektowanie elektronicznego krokomierza, który umożliwiałby pomiary aktywności ruchowej użytkownika, archiwizację tych danych oraz przedstawienie ich w postaci liczby kroków i tempa poruszania się. Mój projekt został zbudowany w oparciu na bardzo popularnym i niezawodnym mikrokontrolerze ESP32 oraz precyzyjnym czujniku inercyjnym MPU6050, co pozwoliło na zaawansowaną analizę sygnałów ruchowych. Funkcjonalność urządzenia rozszerzono o bezprzewodową transmisję danych do serwera WWW, gdzie następuje ich przedstawienie w formie czytelnych wykresów oraz wyświetlacz OLED, który posłużył jako bezpośredni panel informacyjny i uprościł proces odczytu najistotniejszych danych bez potrzeby wchodzenia na stronę internetową. Projekt krokomierza w założeniu miał być zasilany bateryjnie lub przez powerbanka, co umożliwia użytkowanie go bez konieczności ciągłego podłączenia do zewnętrznego źródła zasilania. Projekt obudowy został wykonany w programie Autodesk Fusion, a wydrukowany został przy pomocy drukarki Bambu Lab Studio.

Rozdział 1

Część opisowa

1.1 Analiza rozwiązań rynkowych

Na dzisiejszym rynku dostępna jest szeroka gama urządzeń do monitorowania aktywności fizycznej, które można podzielić na trzy główne kategorie. Zrozumienie ich ograniczeń jest kluczowe dla uzasadnienia budowy systemu opartego na mikrokontrolerze ESP32. Klasyczne krokomierze samodzielne stanowią najprostszą grupę urządzeń. Są to zazwyczaj produkty masowe, wyposażone w podstawowe akcelerometry oraz proste wyświetlacze LCD. Ich funkcjonalność ogranicza się do zliczania kroków w oparciu o sztywne progi amplitudy, bez możliwości archiwizacji danych czy komunikacji bezprzewodowej. Choć cechują się bardzo niskim zużyciem energii, ich zamknięta budowa całkowicie uniemożliwia jakąkolwiek modyfikację algorytmów [1].

Większą precyzję oferują opaski sportowe typu Xiaomi Mi Band oraz smartwatche. Są to zaawansowane produkty komercyjne, wykorzystujące zintegrowane układy inercyjne takie jak akcelerometry 3-osiowe, żyroskopy, a w droższych modelach również GPS. Urządzenia te oferują rozbudowaną wizualizację danych poprzez dedykowane aplikacje mobilne oraz archiwizację wyników w chmurach obliczeniowych producenta. Komunikacja odbywa się głównie za pośrednictwem protokołu Bluetooth Low Energy. Mimo wysokiej precyzji, rozwiązania te są hermetyczne. Użytkownik ma dostęp jedynie do końcowych wyników, nie mając wglądu w surowe dane ani wpływu na proces ich filtrowania [2].



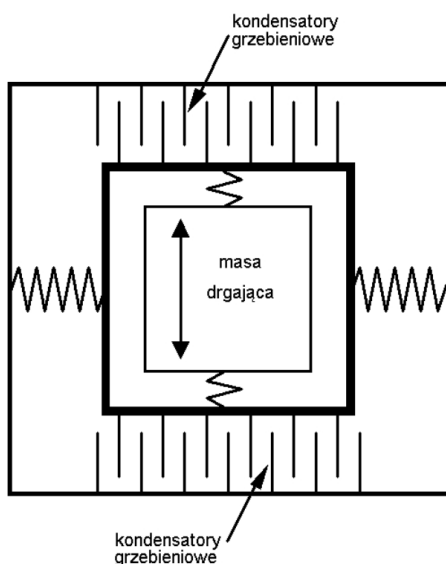
Rys. 1 Inteligentna opaska Xiaomi Mi Band 4 [2]

W tym kontekście zaprojektowany krokomierz stanowi pomost między prostotą urządzeń dedykowanych a zaawansowaniem systemów wearable, wprowadzając jednocześnie unikalną wartość dodaną. Wykorzystanie modułu MPU6050 pozwala na dostęp do precyzyjnych danych inercyjnych, które w przeciwieństwie do innych rozwiązań rynkowych są przetwarzane przez algorytm progowy o otwartej strukturze. Istotną zaletą opracowanego projektu jest także implementacja serwera WWW osadzonego wewnątrz mikrokontrolera. Takie rozwiązanie wyzwania wizualizacji i archiwizacji danych znosi potrzebę utrzymywania zewnętrznych baz danych oraz tworzenia aplikacji mobilnej na systemy Android/iOS. W znacznej mierze podnosi poziom prywatności danych użytkownika poprzez eliminację konieczności przesyłania danych do chmury obliczeniowej, jednocześnie zachowując pierwotny cel funkcjonalności. Projekt zapewnia użytkownikowi także dużą elastyczność w możliwości dalszych prac badawczych i eksperymentów nad zwiększaniem precyzji w analizie biomechaniki ruchu.

1.2 Budowa i zasada działania akcelerometrów typu MEMS

Akcelerometr to urządzenie zdolne do obliczania własnego ruchu w przestrzeni. Funkcjonalność akcelerometrów obejmuje pomiar składowej statycznej przyspieszenia ziemskiego, co pozwala na dokładne wyznaczenie kąta nachylenia obiektu względem wektora grawitacji. Ponadto sensor reaguje na wszelakie zmiany składowej dynamicznej wywoływane ruchem, uderzeniami lub drganiami o niskiej amplitudzie i częstotliwościach do kilkudziesięciu Hz. Przy pomiarze wibracji akcelerometr jest na sztywno mocowany do badanego obiektu, co pozwala na bezpośrednią transmisję drgań na strukturę sensora. Urządzenie to pełni rolę przetwornika elektromechanicznego, konwertując energię kinetyczną drgań na sygnał elektryczny, którego amplituda jest w każdej chwili proporcjonalna do chwilowego przyspieszenia drgającego elementu. Aktualnie najpowszechniejszymi rozwiązaniami są akcelerometry 3-osiowe, które w istocie składają się z trzech osobnych akcelerometrów, z których każdy mierzy przyspieszenie w jednostce g w innej osi – X, Y i Z. Czujnik może zmierzyć przyspieszenie z wartością ujemną, jeśli okazałoby się, że przyspieszenie w jakiegokolwiek z płaszczyzn zadziała w przeciwnym kierunku do ustawienia urządzenia. Zasada działania 3-osiowego akcelerometru w warunkach statycznych opiera się na detekcji wektora grawitacji. W zdefiniowanym położeniu spoczynkowym, gdy na akcelerometr nie działają żadne zewnętrzne przyspieszenia, a oś Y jest skierowana pionowo w dół, składowe przyspieszenia wynoszą $X=0$, $Y=1$, $Z=0$. Analogicznie jak odchylimy akcelerometr w lewo lub w prawo to otrzymamy $X=1$, $Y=0$, $Z=0$ lub $X=-1$, $Y=0$, $Z=0$. Dzięki tym parametrom i dalszym wykorzystaniem ich przez algorytmy można kontrolować pracę akcelerometru. Metodyka działania

oparta jest na ciężarze zamontowanym na sprężynach, gdzie jeden jej koniec jest montowany na stałe do okładzin kondensatora grzebieniowego, a drugi koniec jest przytwierdzony do zamontowanego ciężaru. Pod wpływem przyspieszenia, ciężar zaczyna się przemieszczać na sprężynach, skutkuje to wahaniami odległości pomiędzy elementem kondensacyjnym a masą, co wpływa na zmiany pojemności. Do kluczowych ograniczeń akcelerometrów typu MEMS możemy zaliczyć relatywnie małą dokładność pomiarową, szczególnie w zakresie wysokich amplitud i częstotliwości, a więc sygnałów o wysokiej dynamice. Znaczący poziom szumów własnych oraz nieliniowość charakterystyki w ekstremalnych warunkach dyskwalifikuje je z użycia w systemach bezpieczeństwa przemysłowego, gdzie wymagana jest powtarzalność i precyzja odczytu, choć doskonale sprawdzają się w systemach elektroniki użytkowej [4].



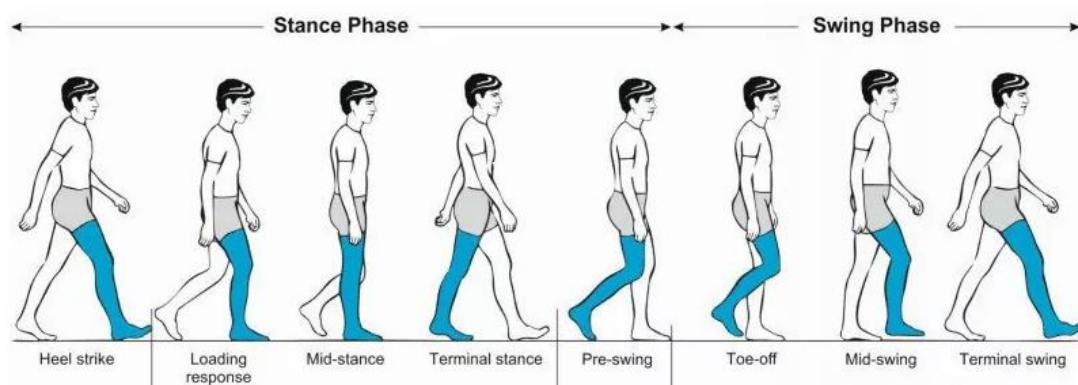
Rys. 2 Grafika przedstawia działanie żyroskopu w technologii MEMS [5]

1.3 Charakterystyka sygnału przyspieszenia podczas chodu człowieka

Rejestrowany podczas chodu sygnał przyspieszenia charakteryzuje się cyklicznością i trójwymiarowością, co wprost wynika z biomechaniki ruchu ludzkiego ciała. Przemieszczanie wiąże się ze zmianą środka ciężkości, które można zaobserwować jako oscylacje w układzie trzech osi ortogonalnych – pionowej, bocznej i przednio-tylnej. Zjawisko to stanowi odzwierciedlenie nieustannych i naprzemiennych faz podporu i wymachu kończyn dolnych, co generuje unikalny wzorec motoryczny. Sygnał przyspieszenia jest niesymetryczny w fazie jednego kroku, lecz jednocześnie charakteryzuje się zachowaniem wysokiej symetrii i powtarzalności w skali długofalowej, co jest efektem naprzemiennego i powtarzalnego charakteru chodu. Kompletny cykl chodu składa się z fazy podporu (stance phase), która zajmuje około 60 procent czasu trwania cyklu,

oraz fazy przenoszenia (swing phase), zajmującej pozostałe 40 procent cyklu. Każda z nich w specyficzny sposób modeluje sygnał inercyjny [6].

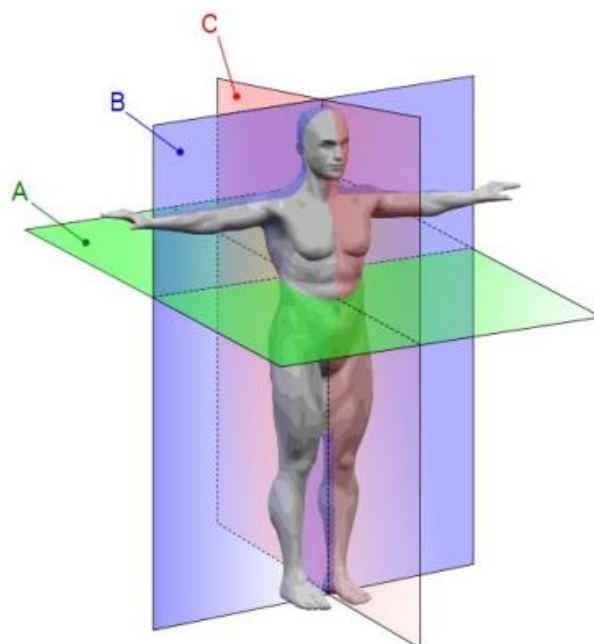
Fazy chodu



Rys. 3 Zdarzenia oraz fazy w cyklu chodu człowieka [6]

Etap kontaktu stopy z podłożem, stanowiący początek fazy podporu, wywołuje silne wymuszenie o charakterze uderowym pozwalające na precyzyjną identyfikację momentu rozpoczęcia nowej sekwencji ruchu. W dalszej kolejności następuje faza amortyzacji, w której stopa przylega równolegle do podłoża, co w danych pomiarowych skutkuje obniżeniem amplitudy przyspieszenia. Środkowa faza podporu, wykorzystuje mechanizm kołyski stawu skokowego do efektywnego przemieszczenia tułowia w przód i wizualizowana jest w sygnale jako okresowa stabilizacja składowych ortogonalnych. Ostatni etap w fazie podporu przypada na okres podwójnego podparcia i przenoszenia środka masy na kończynę przeciwną. Procesowi temu towarzyszy określona zmiana kątowa. Charakterystycznym parametrem geometrycznym tego etapu jest zgięcie stawu kolanowego o 40° i stawu skokowego o 15° , co ostatecznie przerywa kontakt stopy z nawierzchnią [7].

Faza przenoszenia zostaje zainicjowana poprzez dynamiczne odepchnięcie stopy od nawierzchni co nadaje ciału pęd w przód. Efektem jest pojawienie się przyspieszenia o zwrocie zgodnym z wektorem prędkości. W środkowej fazie przenoszenia kończyna mija linię pionową tułowia, a kość piszczelowa przyjmuje na moment pozycję prostopadłą do podłoża. Cały cykl zamyka końcowa faza, w której następuje celowe spowolnienie ruchu kończyny podczas przygotowywania się do kontaktu pięty z nawierzchnią, co pozwala a zainicjowanie kolejnego cyklu pomiarowego [7].



Rys. 4 Płaszczyzny ciała człowieka [7]

Sygnał rejestrowany w płaszczyźnie pionowej odwzorowuje okresowe wahania energii potencjalnej i kinetycznej ciała, co jest bezpośrednią konsekwencją poszczególnych faz chodu. Płaszczyzna przednio-tylna dominuje pod względem przyspieszeń związanych z dynamiką wymachów i hamowań w trakcie amortyzacji. Najmniejszą amplitudę odnotowuje się w płaszczyźnie bocznej, której głównym zadaniem jest monitorowanie stabilności ruchu. Właściwością sygnału chodu jest dążenie do redukcji zbędnych oscylacji i minimalizowania ruchów pasożytniczych, aby osiągnąć jak najmniejszy wydatek energetyczny. Należy jednak zwrócić uwagę, że dynamika tych przebiegów zmienia się wraz ze zmianą tempa chodu lub osobistych cech użytkownika. Projektowany algorytm musi więc z tego względu precyzyjnie identyfikować i rozpoznawać fazy cyklu, aby zachować wiarygodność pomiarową [7].

1.4 Interfejsy komunikacyjne w systemach IoT

Architektura systemów IoT opiera się na interfejsach komunikacyjnych, które stanowią kluczowe ogniwo, umożliwiając współpracę wielu urządzeń wbudowanych. Gromadzą one dane pomiarowe, przetwarzają je lokalnie i przesyłają do innych elementów systemu. Rola interfejsów komunikacyjnych w systemach IoT sprowadza się do pośredniczenia w wymianie danych pomiędzy czujnikami, jednostką centralną oraz infrastrukturą sieciową. Wybór pomiędzy przewodowymi a bezprzewodowymi standardami komunikacji zależy od specyficznych wymagań użytkownika

takich jak przepustowość łącza, wymogi dotyczące energooszczędności, czy zasięg działania urządzenia [8].

1.4.1 Magistrala I²C

Standard I²C będący synchroniczną, dwukierunkową magistralą szeregową, jest bardzo powszechnym sposobem komunikacji w systemach wbudowanych. Wykorzystuje dwie linie sygnałowe – linia SDA jest odpowiedzialna za przepływ danych, podczas gdy linia SCL dostarcza sygnał zegara. Standard I²C umożliwia komunikację wielu urządzeń podrzędnych nazywanych slave, z jednym lub wieloma urządzeniami nadrzędnymi określanymi jako master. Identyfikacja poszczególnych urządzeń odbywa się poprzez przypisane im unikatowe adresy sprzętowe. Początek transmisji wyznacza sygnał start, który jest inicjowany przez opadające zbocze linii SDA, gdy jednocześnie linia SCL jest w stanie wysokim. Gdy warunek zostanie spełniony zaczyna się transmisja danych, stan SCL może ulegać zmianom tylko gdy linia SDA jest w stanie niskim. Odbiornik odczytuje linie SDA w czasie dodatniego impulsu na linii SCL. Przesyłanie bitów następuje w kolejności od najstarszego do najmłodszego. Wadami tego rozwiązania są możliwe zakłócenia przy większych długościach przewodów oraz ograniczona prędkość transmisji. W niniejszej pracy magistrala została wykorzystana do komunikacji między ESP32 a urządzeniami peryferyjnymi – wyświetlaczem OLED oraz akcelerometrem MPU6050 [9].

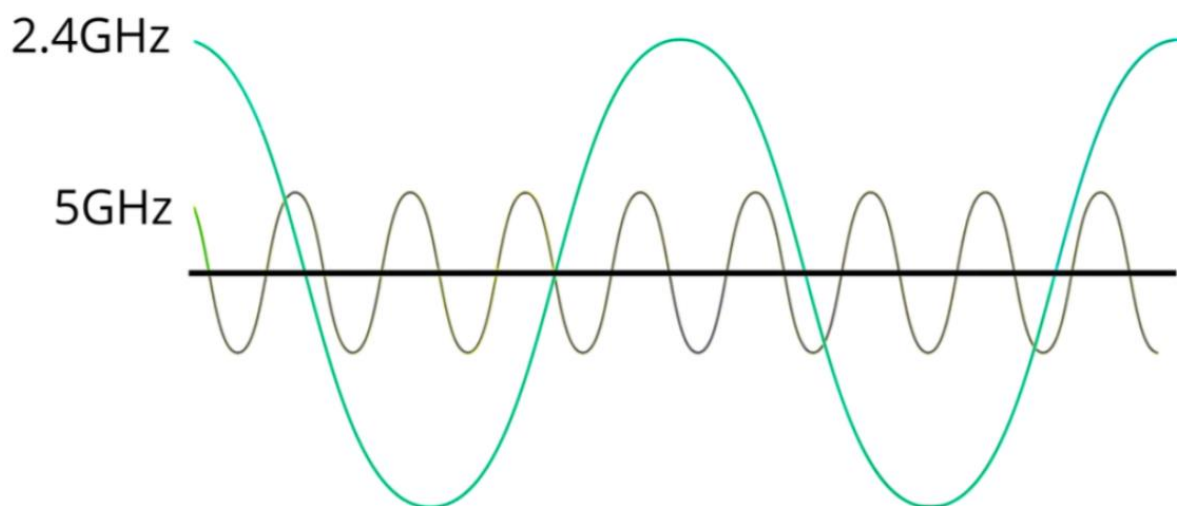
1.4.2 Wi-Fi w systemach wbudowanych

Jednym z głównych i najpopularniejszych standardów komunikacji bezprzewodowych jest Wi-Fi. Jest szczególnie często stosowany w systemach IoT wymagających dużej przepustowości i bezpośredniego dostępu do sieci Internet. Umożliwia komunikację z serwerami HTTP oraz transmisję danych w sieciach WLAN. Mikrokontroler ESP32, którego użyłem w projekcie posiada wbudowany moduł Wi-Fi, który umożliwia mu działanie jako serwer WWW, co znacząco ułatwia projektowanie urządzeń IoT. Komunikacja Wi-Fi opiera się na przyjętych standardach sieciowych IEEE 802.11. Definiują one metody modulacji, jak i parametry fizyczne medium transmisyjnego [10].

Nazwa generacji sieci	Nazwa standardu od 2018	Rok wprowadzenia na rynek	Częstotliwość działania	Maksymalna teoretyczna przepustowość
802.11b	Wi-Fi 1	1999	2,4 Ghz	11 Mb/s
802.11a	Wi-Fi 2	1999	5 Ghz	54 Mb/s
802.11g	Wi-Fi 3	2003	2,4 Ghz	54 Mb/s
802.11n	Wi-Fi 4	2009	2,4 oraz 5 Ghz	600 Mb/s
802.11ac	Wi-Fi 5	2014	2,4 oraz 5 Ghz	1 Gb/s
802.11ax	Wi-Fi 6	2019	2,4 oraz 5 Ghz (docelowo między 1 a 6 Ghz)	10 Gb/s

Rys. 5 Ewolucja standardów Wi-Fi IEEE 802.11 [11]

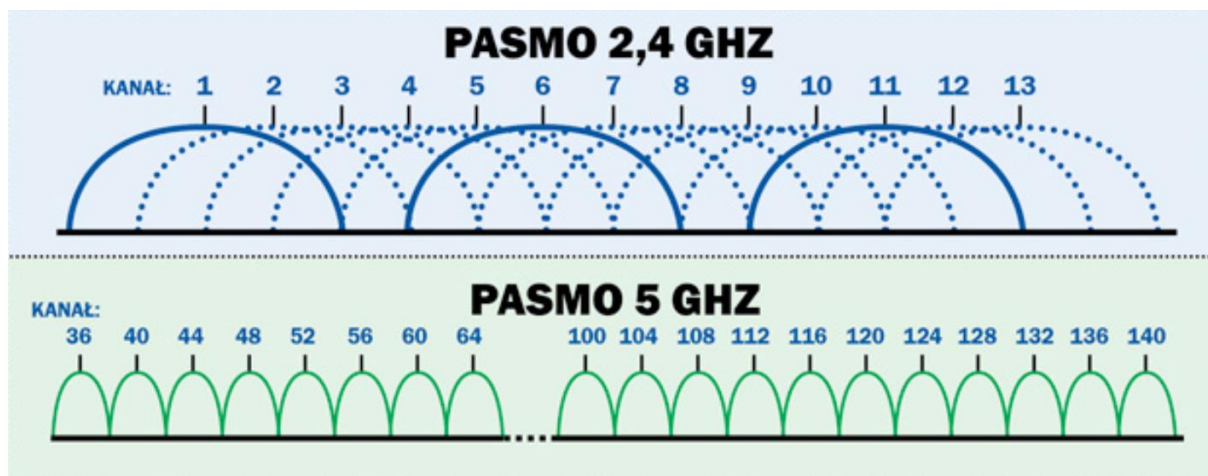
Użyte w projekcie ESP32 obsługuje standardy IEEE 802.11 b/g/n co znacząco ułatwiło integrację z domowymi sieciami bezprzewodowymi [12].



Rys. 6 Obrazowe przedstawienie różnicy w częstotliwości [13]

Technologia Wi-Fi opiera się na wykorzystaniu fal elektromagnetycznych, w konkretnych zakresach częstotliwości, spośród których standardowo wykorzystuje się pasma 2,4 GHz lub 5GHz. Każde z tych pasm oferuje inną specyfikację techniczną i wpływa na zasięg połączenia, odporność na zakłócenia, przepustowość czy efektywność energetyczną. Wykorzystanie częstotliwości 2,4 GHz zapewnia korzystne parametry propagacyjne i charakteryzuje się wysoką przenikalnością przez bariery fizyczne i utrzymanie łączności na większym dystansie niż w przypadku skorzystania z pasma 5 GHz. Z drugiej strony użytkownik musi liczyć się z niską odpornością na szумы radiowe i ryzykiem kolizji pakietów czy zakłóceń, ponieważ zakres ten jest współdzielony przez liczne systemy i akcesoria peryferyjne, co może negatywnie wpływać na stabilność połączenia. Pasma

5 GHz oferuje znaczną przewagę w zakresie przepustowości transmisji i odporności na zakłócenia, z powodu mniejszego zagęszczenia urządzeń pracujących w tym zakresie i większej liczby dostępnych kanałów radiowych. Wyższe częstotliwości umożliwiają transfer dużych ilości danych w krótkim czasie, ale słabiej radzą sobie z przeszkodami takimi jak ściany czy konstrukcje budynków. Ponadto pracowanie w tym zakresie wiąże się z większym zapotrzebowaniem na energię, co stanowi istotny problem dla urządzeń o restrykcyjnym budżecie energetycznym [14].



Rys. 7 Porównanie kanałów transmisyjnych [15]

Dla potrzeb projektowanego krokomierza zastosowanie pasma 2,4 GHz jest w pełni wystarczające i optymalne zważając na niewielkie zapotrzebowanie na transfer danych. System generujący jedynie pakiety z wynikami pomiarowymi nie odniósłby korzyści z większej przepustowości. Jednocześnie większy zasięg w paśmie 2,4 GHz oraz niższe zużycie energii sprzyja wydłużeniu czasu pracy na zasilaniu bateryjnym przy zachowaniu niezawodnego połączenia.

Rozdział 2

Projekt i realizacja urządzenia

2.1 Założenia funkcjonalne i techniczne

Głównym celem projektowym było skonstruowanie krokomierza elektronicznego pozwalającego na monitorowanie aktywności fizycznej w postaci pomiaru liczby kroków i obliczania tempa chodu oraz wizualizacja tych danych dla użytkownika. Urządzenie miało działać przy wykorzystaniu zasilania bateryjnego lub powerbanka i zapewniać stabilną i niezawodną pracę w warunkach codziennego użytkowania. Skonstruowany krokomierz miał umożliwiać detekcję kroków użytkownika na podstawie algorytmu, który przetwarza dane pomiarowe z czujnika inercyjnego, przy jednoczesnym zachowaniu odporności na szумы i zakłócenia czy fałszywe detekcje wynikające z drgań. Wizualizacja pomiarów miała odbywać się przy pomocy wbudowanego w ESP32 serwera HTTP. Serwer wysyła pliki strony internetowej, a przeglądarka po odebraniu danych w postaci JSON miała je odczytać i przedstawić w przystępnej dla użytkownika formie. Projekt zakłada wykorzystanie pamięci nieulotnej Flash do przechowywania firmwaru oraz rekordów kroków użytkownika, co ochrania historię aktywności przed skutkami restartu urządzenia. Z punktu widzenia konstrukcyjnego założono, aby urządzenie było lekkie i kompaktowe. Przyjęta koncepcja zakłada implementację obudowy wykonanej w technologii druku 3D, a jej wymiary nie powinny przekraczać 90 x 60 x 30 mm, a masa całkowita urządzenia nie powinna być większa niż 0,4 kg. Takie parametry zapewniają wysoką mobilność urządzenia i umożliwiają przymocowanie krokomierza do paska spodni, aby był łatwy w użytkowaniu podczas spaceru. Dla zwiększenia wygody użytkownika, urządzenie powinno być wyposażone w wyświetlacz, który byłby umiejscowiony na wierzchniej części obudowy, pozwalając na natychmiastową weryfikację postępów aktywności bez potrzeby każdorazowego łączenia się z serwerem WWW. Powyższe założenia projektowe stanowiły podstawę do dalszych prac.

2.2 Dobór komponentów sprzętowych

Proces doboru podzespołów elektronicznych był kluczowym etapem realizacji projektu, ponieważ ma to wpływ na końcową funkcjonalność, niezawodność oraz energooszczędność projektowanego krokomierza. Podczas selekcji komponentów kierowano się ich wzajemną kompatybilnością, wymaganiami technicznymi, ceną oraz łatwością ich implementacji

w standardzie I²C. W poniższych podrozdziałach wybór poszczególnych elementów składowych zostanie kompleksowo opisany i uzasadniony.

2.2.1 Jednostka centralna ESP32 Dev Kit V1

Do realizacji projektu wybrałem mikrokontroler ESP32 Dev Kit V1 o wadze 6,98 gram i wymiarach 55 x 28 x 8 mm, który stanowi popularną i szeroko dostępną platformę rozwojową. Wybór tego mikrokontrolera podyktowany był przede wszystkim jego wszechstronnością, wysoką wydajnością obliczeniową, niskim poborem mocy oraz korzystnym stosunkiem ceny do funkcjonalności co ma istotne znaczenie w projektach prototypowych i edukacyjnych [16].

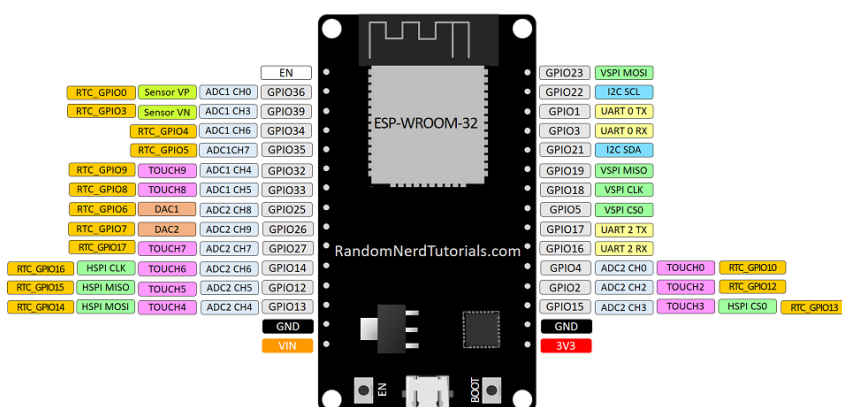


Rys. 8 Moduł ESP32 Dev Kit V1[16]

Jednostka centralna ESP32 jest oparta na dwurdzeniowym procesorze Tensilica LX6 o taktowaniu 240 MHz, oferując wysoką moc obliczeniową potrzebną do wielozadaniowej pracy systemu. Układ dysponuje 520 kB pamięci SRAM oraz 4 MB pamięci nieulotnej Flash. Płytką oferuję 30 wyprowadzeń GPIO co znacząco ułatwia integrację komponentów peryferyjnych. Dodatkowo moduł wspiera standardowe interfejsy komunikacyjne – I²C, SPI oraz UART, co zapewnia niezawodną wymianę danych z modułami zewnętrznymi. W opisanym projekcie interfejs I²C pełni rolę głównej magistrali komunikacyjnej do obsługi czujnika inercyjnego MPU6050 oraz wyświetlacza OLED [16].

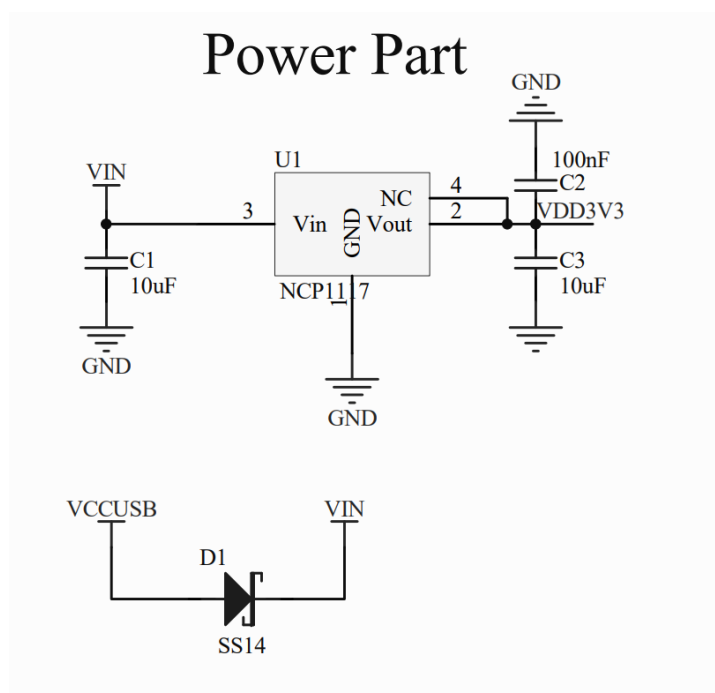
ESP32 DEVKIT V1 – DOIT

version with 30 GPIOs



Rys. 9 Schemat wyprowadzeń pinów na płytce ESP32 Dev Kit V1 [17]

Łączność sieciową w projekcie zapewnia wbudowany moduł Wi-Fi, pracujący w standardzie 802.11 b/g/n (pasmo 2,4 GHz). Bezpieczeństwo połączenia i transmisji danych realizowane jest przez zastosowanie zaawansowanych mechanizmów zabezpieczeń, takich jak WPA. Oprócz Wi-Fi mikrokontroler posiada również moduł Bluetooth v4.2 oraz Bluetooth Low Energy, co stwarza szerokie perspektywy do przyszłej rozbudowy projektu o współpracę z aplikacją mobilną [16].



Rys. 10 Schemat sekcji zasilania ESP32 Dev Kit V1 [18]

Płytkę jest wyposażoną w zintegrowany stabilizator napięcia typu LDO, a dokładniej w widoczny na powyższym schemacie model NCP1117 stosowany zamiennie z układem ASM1117. Jego kluczową funkcją jest obniżenie napięcia wejściowego V_{IN} do stabilnego poziomu 3,3 V, wymaganego przez mikrokontroler ESP32. Zastosowany zestaw kondensatorów filtrujących

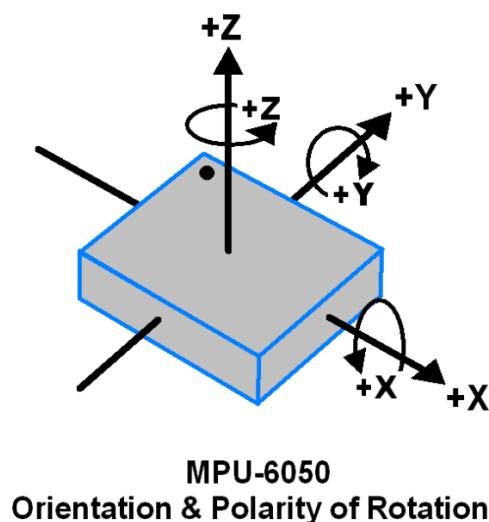
C_1 , C_2 , C_3 o pojemnościach 10 μF i 100 μF , ma za zadanie wygładzenie tętnień zasilania przy nagłych skokach poboru prądu oraz eliminację zakłóceń wysokoczęstotliwościowych. Dodatkowym elementem ochronnym uzupełniającym układ zasilania jest dioda zabezpieczająca D_1 , włączona szeregowo między V_{CCUSB} a V_{IN} . Jest to dioda Schottky-ego typu SS14, a jej głównym zadaniem jest zabezpieczenie przed powstaniem prądu zwrotnego oraz ochrona portu USB komputera w przypadku pojawienia się wyższego napięcia na zewnętrznym pinie zasilania [19].

System zasilania modułu został zaprojektowany elastycznie, aby zasilanie można było dostarczyć poprzez złącze micro USB lub pin V_{IN} . Jeżeli na pin V_{IN} podamy za duże napięcie wtedy zostanie wykorzystany wbudowany układ stabilizacji napięcia, co w efekcie obniży napięcie do preferowanego przez mikrokontroler napięcia zasilania 3,3 V i zapewni bezpieczne warunki pracy. Alternatywnym sposobem zasilania jest bezpośrednie podanie napięcia 3,3 V na odpowiedni pin płytki (3V3). W tym przypadku napięcie to ominie wbudowany stabilizator LDO, co oznacza brak dodatkowej ochrony przed ewentualnym przepięciem. Wymusza to zastosowanie stabilizowanego i precyzyjnego źródła zasilania, zapewniającego napięcie w bezpiecznym zakresie od 3,0 V do 3,6 V. Zignorowanie zalecanych parametrów zasilania może skutkować częstymi błędami w funkcjonalności kodu, niestabilnej pracy, a przy przekroczeniu górnej bariery 3,6 V do trwałego uszkodzenia modułu [19].

Zastosowanie mikrokontrolera ESP32 Dev Kit V1 jest uzasadnione nie tylko przez jego akceptowalne gabaryty, ale też dzięki szerokiemu wsparciu programistycznemu w środowisku Arduino IDE. Powyższy moduł idealnie nadaje się do wykorzystania w systemach pomiarowych IoT, takich jak zaprojektowany krokomierz elektryczny.

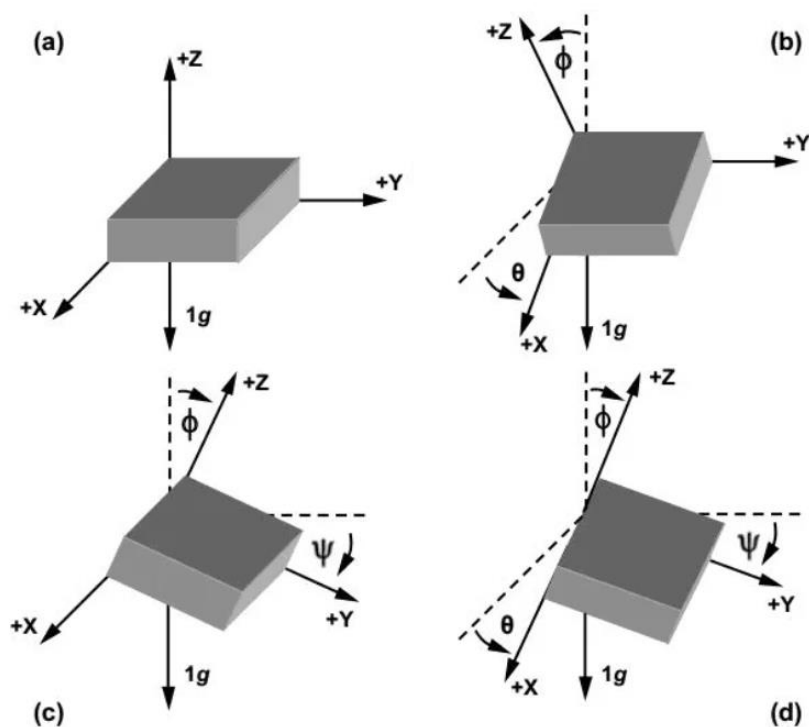
2.2.2 Moduł pomiarowy MPU 6050

Sercem układu pomiarowego jest moduł MPU 6050, jest to kompletne rozwiązanie do śledzenia ruchu, które integruje w sobie trójosiowy akcelerometr oraz trójosiowy żyroskop. Dzięki temu cały moduł umożliwia sześciooosiową detekcję ruchu i pozwala na jednoczesny pomiar przyspieszeń liniowych jak i rotacji przestrzeni trójwymiarowej. Skutkuje to bardzo precyzyjnym wykrywaniem zmian w ruchu, rotacji jak i orientacji urządzenia i czyni z tego czujnika idealny komponent do aplikacji związanych z monitorowaniem aktywności fizycznych związanych z analizą ruchu. Dodatkową funkcjonalnością jest wbudowany czujnik temperatury. Całkiem istotną rolę podczas wyboru czujnika odgrywała również przystępna cena [20].



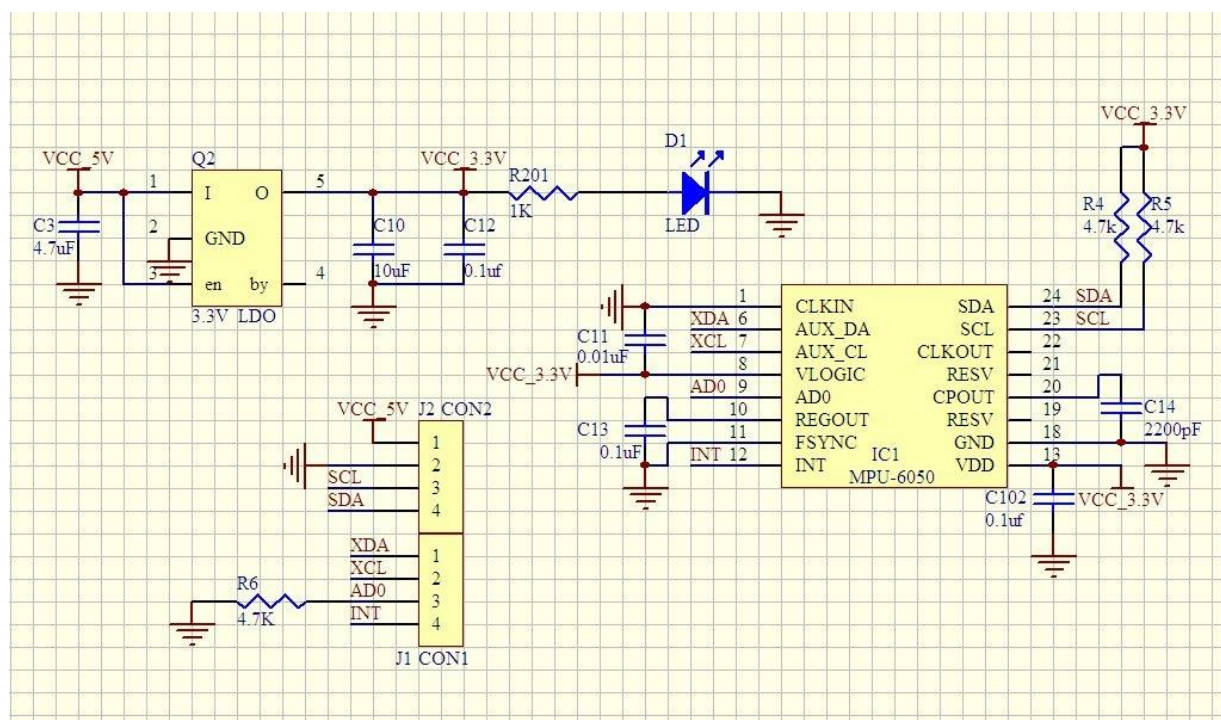
Rys. 12 Grafika przedstawia trzyosiowy żyroskop [21]

Trójosioowy akcelerometr stanowi kluczowy komponent w module MPU 6050. Ma za zadanie dokonać pomiaru przyspieszeń linowych oraz kąta nachylenia w trzech osiach X, Y i Z. Mechanizm funkcjonowania czujnika bazuje na pomiarze przyspieszenia masy ruchomej na pod wpływem sił bezwładności, co prowadzi do niezbilansowania układu kondensatorów pomiarowych i generuje sygnał elektryczny o amplitudzie proporcjonalnej do działającego przyspieszenia. Wynikowe napięcie jest następnie konwertowane na postać cyfrową za pomocą 16-bitowych przetworników ADC. Dzięki swojej precyzji, czujnik potrafi wykryć pełną wartość przyciągania ziemskiego. Gdy moduł leży w stanie spoczynku na płaskiej powierzchni czujnik pokaże wartość $+1g$ dla osi Z i $0g$ dla osi X oraz Y [21].



Rys. 13 Orientacja osi pomiarowych i kątów obrotu modułu MPU 6050 [21]

Moduł posiada wbudowany procesor DMP, co pozwala na przeniesienie złożonych obliczeń bezpośrednio do układu MPU 6050 i odciąża główny procesor ESP32. Wykorzystanie interfejsu I²C pozwala na komunikację z mikrokontrolerem. Wysoka odporność na przeciążenia sięgająca 10 000 g oraz niski pobór prądu w granicach 5 mA czynią ten moduł uzasadnionym wyborem do urządzeń noszonych, takich jak projektowany krokomierz [21].



2.2.3 Wyświetlacz OLED

W projektowanym krokomierzu jako podstawę interfejsu użytkownika wykorzystałem wyświetlacz wykonany w technologii OLED. Charakteryzuje się wysokim kontrastem i głęboką czernią, co wynika z braku konieczności podświetlania tła – każdy piksel emituje światło samodzielnie. Dzięki temu wyświetlany obraz pozostaje wyraźny i czytelny nawet pod kątem patrzenia powyżej 160°, co znacząco ułatwia odczyt danych. Cena wyświetlacza jest przystępna, a jak we wszystkich wyżej wspomnianych urządzeniach peryferyjnych priorytetem była optymalizacja kosztów przy jednoczesnym zachowaniu wymaganej funkcjonalności [23].

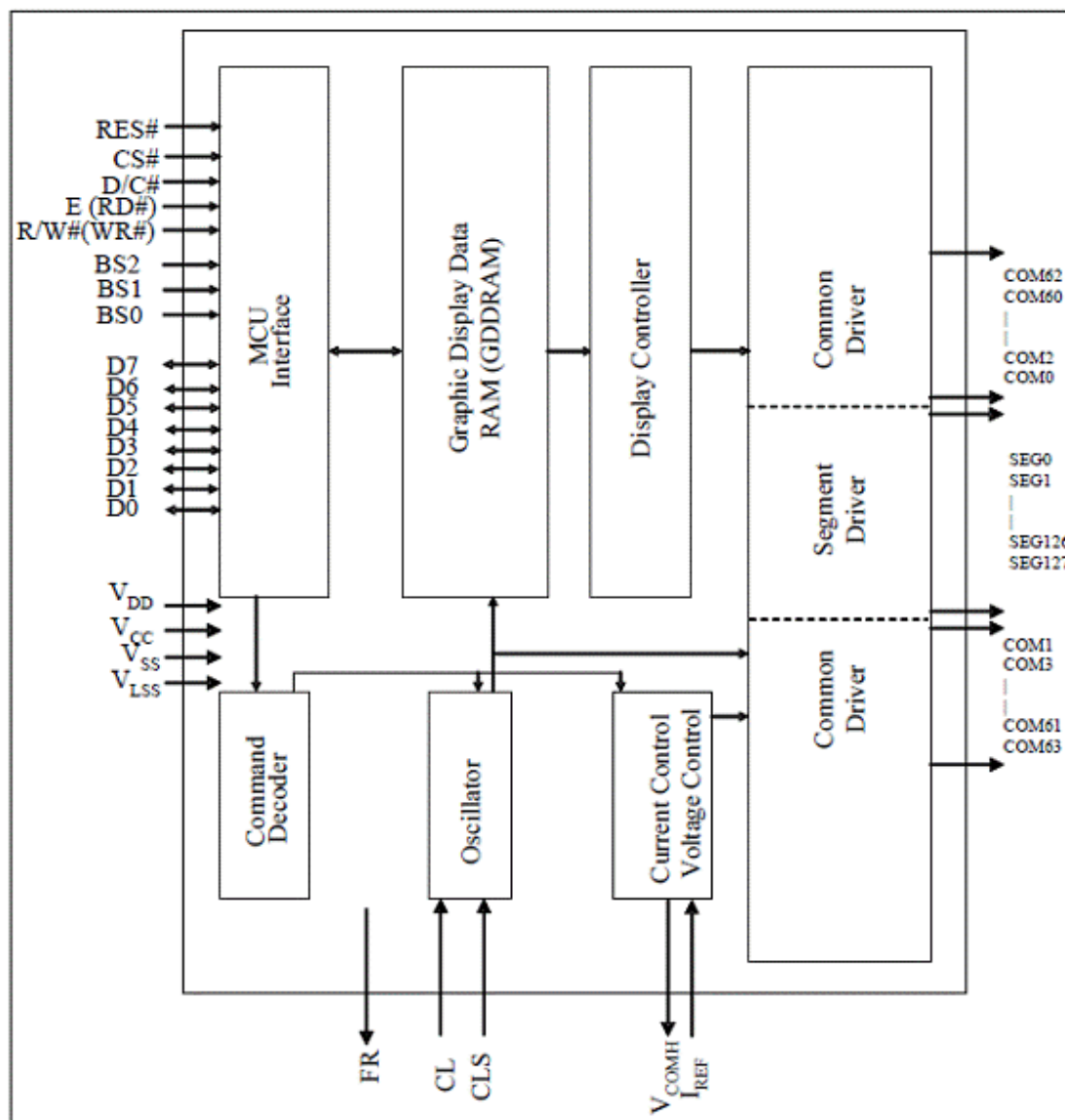


Rys. 15 Moduł wyświetlacza OLED [23]

Użyty w projekcie krokomierza moduł wyświetlacza ma przekątną ekranu 0,96 cala, której aktywna część wynosi 26,4 x 14,7 mm i generuje obraz w kolorze niebieskim. Rozdzielczość ekranu wyświetlacza wynosi 128 x 64 piksele co zapewnia optymalny balans między rozmiarami a czytelnością interfejsu. Daje to możliwość czytelnej prezentacji danych w różnych warunkach oświetlenia [23].

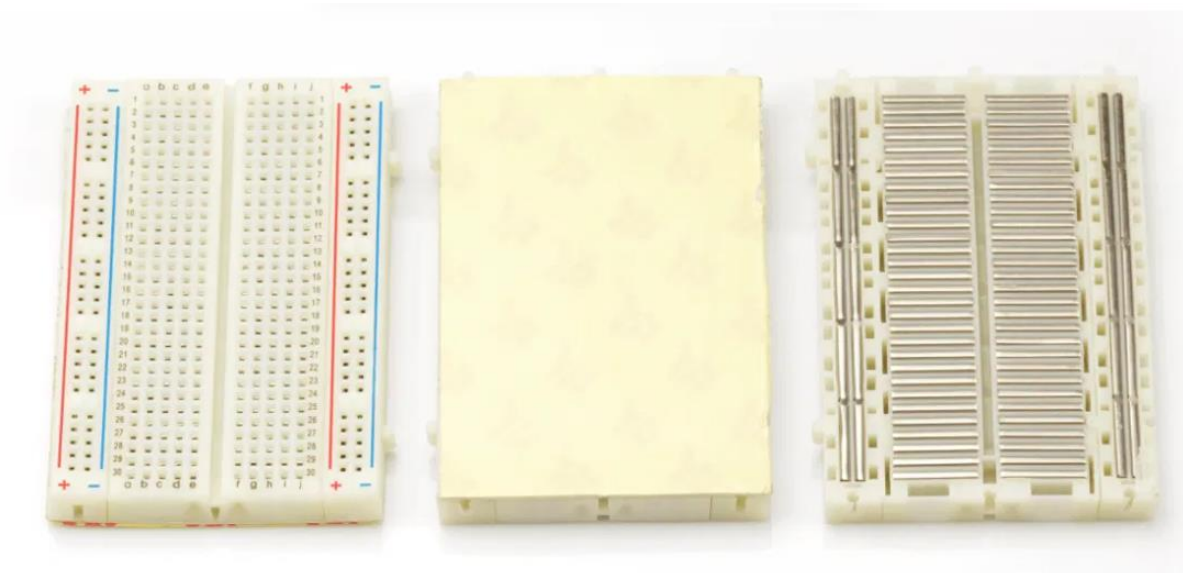
Sercem modułu jest sterownik SSD1306, który ma możliwość komunikowania się z systemem nadrzędnym za pomocą magistrali I²C. Istotnym atutem konstrukcyjnym modułu jest wbudowany stabilizator napięcia LDO. Pozwala on na elastyczne zasilanie układu zarówno napięciem 3,3 V bezpośrednio z ESP32, jak i napięciem 5 V, co zwiększa bezpieczeństwo i uniwersalność całego systemu. Dodatkowo, sterownik SSD1306 charakteryzuje się szerokim zakresem temperatury pracy, co gwarantuje stabilne działanie krokomierza w warunkach zewnętrznych, niezależnie od pory roku [24].

Wybór tego konkretnego modelu niósł ze sobą szereg korzyści wynikających z jego specyfiki technicznej. Przede wszystkim brak konieczności podświetlania tła przekłada się na bardzo niski pobór prądu, co przy stosunkowo niskiej cenie modułu czyni go rozwiązaniem niezwykle efektywnym ekonomicznie i energetycznie. Niewielkie wymiary tego modułu pomogą w zachowaniu kompaktowych wymiarów całego projektu. W projekcie wyświetlacz pełni rolę głównego interfejsu, prezentując w kilku liniach najważniejsze dla użytkownika informacje. Taka forma wizualizacji danych jest niezwykle użyteczna, gdyż pozwala na natychmiastowe sprawdzenie poprawności działania urządzenia oraz bieżący odczyt parametrów aktywności bez konieczności korzystania z zewnętrznych narzędzi diagnostycznych.



2.2.4 Płytki stykowe

W procesie budowy prototypu wykorzystano płytkę stykową, która stanowiła fundament dla połączeń elektrycznych między wszystkimi modułami. Narzędzie to umożliwiło szybkie i bez lutowe łączenie mikrokontrolera ESP32 z czujnikiem MPU6050 oraz wyświetlaczem OLED, co było kluczowe na etapie testów i weryfikacji założeń projektowych. Dzięki wewnętrznej strukturze metalowych szyn, płytka zapewnia pewny styk i stabilne rozproszanie zasilania oraz sygnałów magistrali I²C bez ryzyka trwałego uszkodzenia komponentów przez wysoką temperaturę lutowania. Płytkę ma wymiary 83 x 55 mm, co pozwoliło na budowę kompaktowego prototypu i ułatwiło organizację przewodów w ograniczonej przestrzeni roboczej [25].



Rys. 17 Płytką stykowa [25]

2.2.5 Źródło zasilania mobilnego

W projekcie jako zewnętrzne źródło zasilania wykorzystano powerbank Hama Supreme 20HD o pojemności 20000 mAh. Skorzystano z takiego rozwiązania, ponieważ w złożeniu projekt miał pracować stabilnie na długich trasach terenowych i jednocześnie mieć zapewnioną nieograniczoną mobilność. Zastosowany akumulator litowo-polimerowy charakteryzują się dużą mocą, co przy niskim poborze prądu przez mikrokontroler oraz wyświetlacz OLED zapewnia wielodniowy czas pracy bez konieczności ładowania. Zasilą mikrokontroler oraz pozostałe komponenty peryferyjne napięciem 5V przez standardowy port USB, które następnie jest obniżane do 3,3V przez wewnętrzny stabilizator mikrokontrolera. Urządzenie to stanowi niezawodną platformę energetyczną, gwarantującą stabilność pracy prototypu w różnych scenariuszach testowych [26].



Rys. 18 Powerbank Hama Supreme 20HD [26]

2.3 Montaż urządzenia

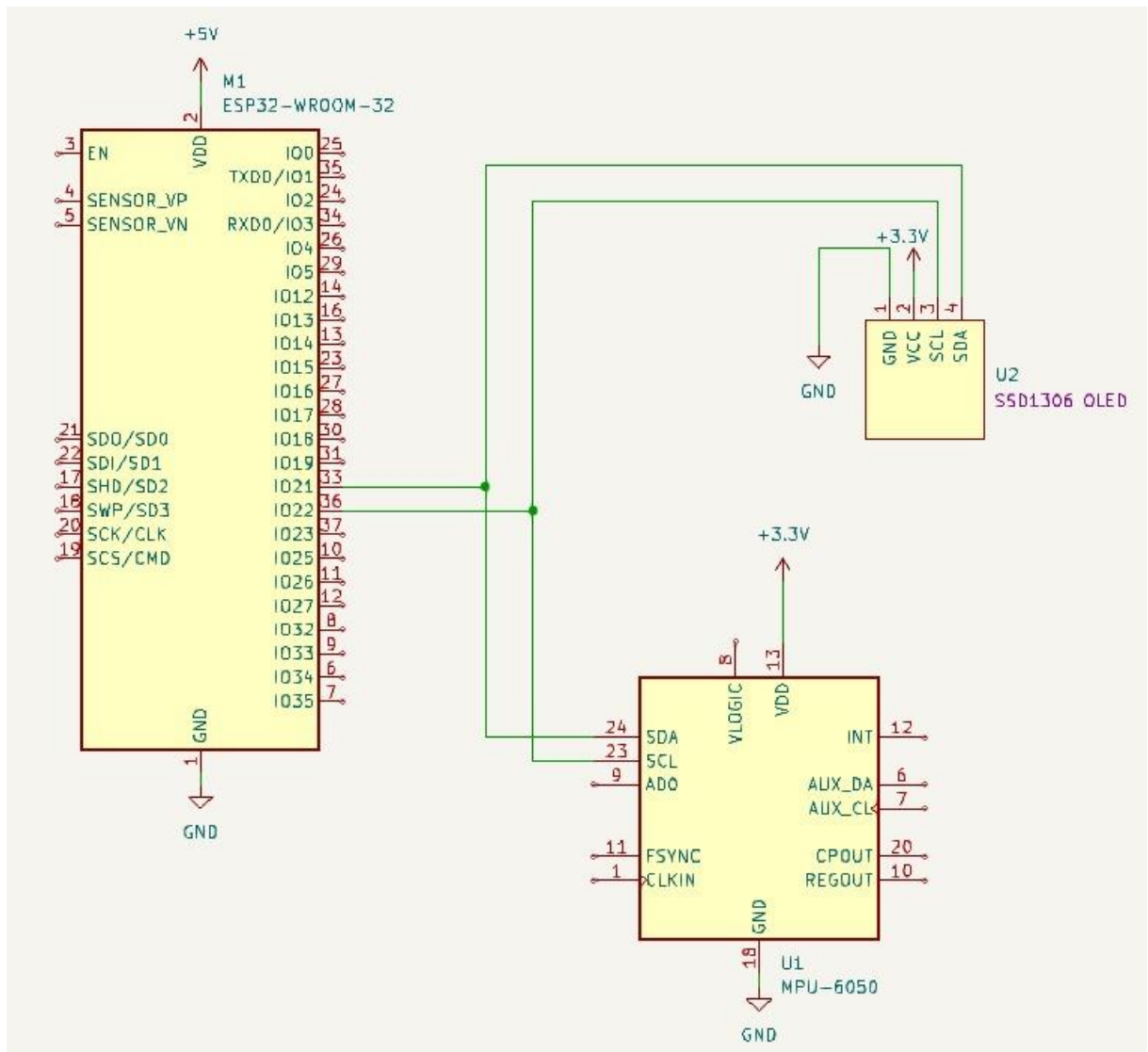
Proces montażu prototypu oparto na metodzie montażu na płytce stykowej, co pozwala na łatwe łączenie komponentów i szybką diagnostykę sygnałów.

Kluczowym elementem montażu jest poprawne połączenie modułów peryferyjnych z jednostką centralną ESP32, do czego wykorzystano magistralę szeregową I²C. Zgodnie z notą katalogową mikrokontrolera, do obsługi komunikacji przypisano dedykowane piny GPIO:

- GPIO 21 – linia danych SDA,
- GPIO 22 – linia zegarowa SCL.

Wyprowadzenia tych pinów przy pomocy przewodów połączeniowych połączono do dwóch osobnych szyn płytki stykowej, do których następnie podpięto piny SCL i SDA z akcelerometru MPU6050 oraz wyświetlacza OLED. Dzięki adresowaniu sprzętowemu urządzeń, mikrokontroler jest w stanie zarządzać obydwoma modułami za pomocą zaledwie dwóch przewodów sygnałowych, co znacząco upraszcza architekturę połączeń.

Cały system zasilany jest za pośrednictwem kabla USB podłączonego do mikrokontrolera. Dostarczone napięcie 5V z portu USB trafia na wbudowany w płytkę ESP32 stabilizator LDO. Konwerter obniża napięcie do stabilnego i bezpiecznego dla reszty komponentów poziomu 3,3V które jest wyprowadzone na pin opisany jako 3V3. Z pinu 3V3 poprowadzono przewód do szyny dodatniej (+) na płytce stykowej, z której zasilane są moduły MPU6050 oraz OLED. Ważnym aspektem całego procesu montażu jest zapewnienie wspólnego punktu odniesienia dla sygnałów elektrycznych. Wszystkie piny GND mikrokontrolera oraz modułów zostały połączone ze sobą na wspólnej szynie ujemnej (-) płytki stykowej, tworząc wspólną masę układu.



Rys. 19 Zrzut ekranu z programu KiCad przedstawiający schemat połączeń komponentów

Po zakończeniu montażu została przeprowadzona kontrola przy pomocy multimetru ustawionego w tryb ciągłości obwodu, pod kątem wykrycia ewentualnych zwarcć. Po podłączeniu zasilania, na mikrokontrolerze oraz module MPU 6050 zapaliły się diody sygnalizujące, co potwierdziło poprawność połączeń.

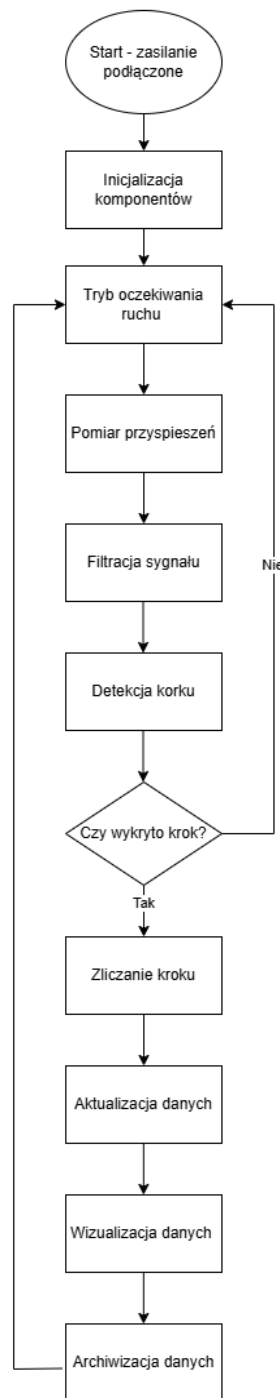
2.4 Algorytm działania oprogramowania

Działanie systemu zaczyna sekwencja inicjalizacyjna, która obejmuje konfigurację portu szeregowego, modułu komunikacyjnego Wi-Fi oraz magistrali I²C potrzebnej do komunikacji z czujnikiem MPU 6050 oraz wyświetlaczem OLED. Na tym etapie następuje również odczyt archiwalnych z pamięci nieulotnej NVS, aby zapewnić ciągłość pomiarów po restarcie.

Realizacja głównej pętli programu obejmuje trzy etapy:

1. Przetwarzanie danych pomiarowych z czujnika inercyjnego,
2. Aktualizacja licznika po pozytywnej detekcji kroku,
3. Wizualizacja i archiwizacja danych.

Na rysunku 20 przedstawiony jest schemat blokowy opisujący logikę działania.



Rys. 20 Schemat blokowy algorytmu działania oprogramowania

2.5 Implementacja oprogramowania

Oprogramowanie stanowi kluczowy element projektu, integrując ze sobą wszystkie komponenty sprzętowe w jeden spójny system. Jego głównym zadaniem jest obsługa komunikacji z czujnikiem inercyjnym oraz wyświetlaczem, a przede wszystkim realizacja algorytmu detekcji i zliczania kroków. Ponadto, program zarządza modułem Wi-Fi mikrokontrolera, umożliwiając prezentację danych za pośrednictwem serwera WWW na ESP32. Szczegółowy opis algorytmów oraz konfiguracji serwera został przedstawiony w dalszych podrozdziałach.

2.5.1 Środowisko programistyczne i wykorzystane biblioteki

Jako głównego narzędzia programistycznego do realizacji projektu wybrałem Arduino IDE, wykorzystujące bibliotekę opartą na języku C++. Jest to bardzo popularne środowisko programistyczne, cechuje się wysoką intuicyjnością i jest dostosowane dla programistów o różnym stopniu zaawansowania. Wybór ten był podyktowany pełnym wsparciem dla układów ESP32 oraz dostępem do szerokiego zbioru bibliotek, które znacząco ułatwiają proces integracji komponentów sprzętowych.

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <MPU6050.h>
#include <math.h>
#include <Preferences.h>
```

Rys. 21 Implementacja bibliotek zrzut ekranu z środowiska Arduino IDE

Poniżej zestawiono i opisano kluczowe biblioteki, które stanowią fundament oprogramowania krokomierza:

WiFi.h oraz WebServer.h umożliwiają zarządzanie i konfigurację wbudowanym w ESP32 modułem Wi-Fi. Dzięki nim można połączyć się z siecią bezprzewodową, następnie stworzyć serwer HTTP oraz udostępniać dane pomiarowe w formacie JSON, które później mogą być odczytane przez przeglądarkę.

Wire.h to biblioteka systemowa która obsługuje protokół komunikacyjny I²C. Jest kluczowa dla synchronizacji pracy mikrokontrolera z czujnikiem inercyjnym MPU 6050 i wyświetlaczem OLED.

Adafruit_GFX.h to zaawansowany pakiet graficzny. Biblioteka ta stanowi podstawę do wyświetlania tekstu i grafiki na wyświetlaczach OLED. Zawiera potrzebne do rysowania funkcje, czcionki i znaki.

Adafruit_SSD1306.h to biblioteka dedykowana do wyświetlacza OLED opartego na sterowniku SSD1306. Odpowiada za komunikację z pamięcią SRAM wyświetlacza.

Implementacja biblioteki MPU6050.h służącej do obsługi akcelerometru, ułatwiła konfigurację rejestrów akcelerometru i żyroskopu. Pozwala na szybki odczyt surowych danych o przyspieszeniach w trzech osiach, które są następnie przetwarzane przez algorytm.

Math.h to klasyczna biblioteka matematyczna wykorzystywana do obliczeń. Jest kluczowa dla algorytmu detekcji kroków i używam jej między innymi do obliczania modułu wektora wypadkowego przyspieszenia, za pomocą pierwiastkowania kwadratów sum składowych.

Preferences.h jest to specjalistyczna biblioteka dla układów ESP32. Zapewnia ona dostęp do nieulotnej pamięci Flash mikrokontrolera. Została wykorzystana do trwałego przechowywania liczby kroków chroniąc przed ich utratą w przypadku nagłego odłączenia zasilania lub restartu urządzenia.

2.5.2 Opracowanie algorytmu detekcji kroków

Aby wykryć krok algorytm analizuje dane pochodzące z akcelerometru w trzech osiach: X, Y oraz Z. Każda z tych osi rejestruje inne składowe ruchy, co pozwala na stworzenie pełnego obrazu aktywności użytkownika. W przypadku osi pionowej Z największe zmiany przyspieszenia są rejestrowane w momencie uderzenia stopy o podłoże. Osie poziome X oraz Y rejestrują ruchy związane z kołysaniem bioder i przesunięciem środka ciężkości ciała do przodu.

Ponieważ urządzenie może być zamocowane pod różnymi kątami, najlepszą i najskuteczniejszą metodą detekcji jest obliczanie wypadkowego wektora przyspieszenia. Pozwala to na uniezależnienie pomiaru od orientacji czujnika. Wartość tę oblicza się ze wzoru:

$$a_{\text{tot}} = \sqrt{(a_x^2 + a_y^2 + a_z^2)}$$

gdzie:

a_{tot} – moduł wektora wypadkowego przyspieszenia całkowitego,

a_x – przyspieszenie liniowe zmierzone w osi X,

a_y – przyspieszenie liniowe zmierzone w osi Y,

a_z – przyspieszenie liniowe zmierzone w osi Z.

```
double axd=ax, ayd=ay, azd=az;
float acc_total = sqrt(axd*axd + ayd*ayd + azd*azd);
```

Rys. 22 Zrzut ekranu z programu Arduino Uno pokazujący wzór przyspieszenia całkowitego, obliczanego ze składowych ax, ay, az akcelerometru

Dzięki temu rozwiązaniu algorytm nie szuka detekcji kroku w jednej konkretnej osi, a bierze pod analizę ogólną zmianę energii ruchu. Typowy cykl kroku charakteryzują się występowaniem wyraźnego ekstremum lokalnego, jeśli dany impuls przekroczy zaprogramowaną wartość progową, zostaje on zarejestrowany w systemie jako pojedynczy krok.

Sygnał jest następnie filtrowany za pomocą filtra wykładniczego, którego działanie można opisać równaniem:

$$a_{\text{filt}(n)} = \alpha \cdot a_{\text{filt}(n-1)} + (1 - \alpha) \cdot a_{\text{tot}(n)}$$

Gdzie α jest współczynnikiem filtra ustalonym eksperymentalnie na 0,8 co zapewnia dobrą równowagę między szybkością reakcji a tłumieniem szumów. Eliminuje nagłe drgania co zapewnia stabilną pracę algorytmu.

```
// filtr responsywny
acc_filtered = 0.8f*acc_filtered + 0.2f*acc_total;
```

Rys. 23 Zrzut ekranu przedstawiający filtr

Najważniejszą częścią programu jest sam algorytm detekcji, w tej części programu następuje decyzja czy ruch zapoczątkowany przez użytkownika był krokiem. Detekcja kroku następuje, gdy jednocześnie są spełnione wszystkie poniższe warunki:

- Przekroczenie progu przyspieszenia całkowitego,
- Wykrycie relatywnie dużej zmiany przyspieszenia składowej a_z ,
- Upłynięcie przynajmniej 400 ms od wykonania ostatniego kroku.

```
bool warunek_kroku = (acc_filtered > 17200.0f) && (acc_diff_z > 1000);
bool odstep = (t - last_step_time > 400);
```

Rys. 24 Zrzut ekranu przedstawiający warunki do detekcji kroku

Jeśli warunki zostały spełnione następuje inkrementacja licznika kroków.

```

if(warunek_kroku && !step_in_progress && odstep){
    krok_counter++;
    step_in_progress = true;
    last_step_time = t;
}

```

Rys. 25 Zrzut ekranu z programu Arduino Uno który pokazuje inkrementację licznika, jeśli spełnione są kryteria kroku

2.5.3 Inicjalizacja systemów łączności i interfejsu wizualnego

Pierwszym etapem działania programu po uruchomieniu mikrokontrolera jest konfiguracja wyświetlacza OLED oraz nawiązanie połączenia z siecią bezprzewodową. To kluczowy proces warunkujący możliwość przesyłania danych do interfejsu użytkownika. W sekcji konfiguracyjnej zdefiniowano parametry sieciowe, takie jak identyfikator SSID oraz hasło dostępu.

```

// KONFIG WIFI
const char* ssid    = "Galaxy S20 FE 5G 19EB";
const char* password = "ttqy5329";
//OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
unsigned long lastOLED = 0;

```

Rys. 26 Konfiguracja danych uwierzytelniających sieci Wi-Fi (SSID i hasło) oraz definicja parametrów wyświetlacza OLED

Po wywołaniu funkcji `WiFi.begin()` system przechodzi w tryb oczekiwania, aż uzyska połączenie z siecią. Po udanej autoryzacji, mikrokontroler pobiera z routera adres IP i wypisuje go w terminalu. Równolegle z procesem sieciowym następuje uruchomienie interfejsu wizualnego. Program sprawdza fizyczną obecność sterownika SSD1306 pod adresem 0x3C. W przypadku braku wykrycia modułu system wstrzymuje dalszą pracę.

```

// Wi-Fi
WiFi.begin(ssid, password);
Serial.print("Łączenie z Wi-Fi");
unsigned long t0 = millis();
while(WiFi.status() != WL_CONNECTED && millis()-t0 < 20000){
  delay(500); Serial.print(".");
}
if(WiFi.status() == WL_CONNECTED){
  Serial.print("\n IP: "); Serial.println(WiFi.localIP());
}else{
  Serial.println("\n Brak połączenia z Wi-Fi (timeout). Spróbuj ponownie.");
}

```

Rys. 27 Zrzut ekranu ze środowiska Arduino Uno przedstawiający proces łączenia się siecią Wi-Fi oraz wyświetlanie adresu IP serwera po nawiązaniu połączenia

```

if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
  Serial.println("OLED nie wykryty");
  while (1) delay(100);
}
display.clearDisplay();
display.setTextColor(SSD1306_WHITE);
display.display();

```

Rys. 28 Proces inicjalizacji wyświetlacza OLED

2.5.4 System archiwizacji danych i serwer WWW

Aby zapewnić użyteczność krokomierza jako urządzenia mobilnego, zaimplementowano system trwałego przechowywania danych oraz mechanizm dynamicznej wymiany informacji z interfejsem użytkownika. Wykorzystałem do tego bibliotekę Preferences.h, która operuje na partycjach NVS pamięci Flash mikrokontrolera.

```

// Archiwizacja
prefs.putInt("kroki", krok_counter);
prefs.putInt("wysw", wyswietlane_kroki);
prefs.putInt("histHead", histHead);

for(int i = 0; i < HISTORY_POINTS; i++){
  prefs.putInt(("h" + String(i)).c_str(), history[i]);
}

Serial.println("Dane zapisane do FLASH");
}

```

Rys. 29 Fragment kodu odpowiadający za archiwizację danych

Zastosowany mechanizm archiwizacji wykonuje zapis w sytuacji wykrycia kroku oraz cyklicznie po zamknięciu 10-sekundowego okna historii. Dzięki temu urządzenie po restarcie odczytuje ostatni stan licznika oraz całą historię aktywności z ostatnich 15 minut. Wymiana informacji między serwerem WWW działającym na ESP32 a przeglądarką opiera się na protokole

HTTP oraz formacie JSON. Po wpisaniu w przeglądarkę adresu IP serwera, przeglądarka wysyła żądanie HTTP GET w celu pobrania plików strony. Serwer nasłuchuje i odpowiada na żądanie, wysyłając dokument tekstowy w formacie JSON zawierający aktualną sumę kroków, wyliczone tempo oraz tablice historyczne potrzebne do narysowania wykresu i odsyła go do przeglądarki. Przeglądarka na podstawie otrzymanych danych dokonuje wizualizacji interfejsu użytkownika w przystępnej formie graficznej.

```
String json = "{";
json += "\"total_steps\":" + String(krok_counter) + ",";
json += "\"pace_per_min\":" + String(sumLastMin) + ",";
json += "\"labels\":" + labels + ",";
json += "\"history\":" + hist;
json += "}";
server.send(200, "application/json", json);
```

Rys. 30 Fragment kodu odpowiedzialny za generowanie i wysyłanie danych w formacie JSON

```
async function fetchData(){
  try{
    const r = await fetch('/data.json', {cache:'no-store'});
    if(!r.ok) throw new Error('HTTP '+r.status);
    const j = await r.json();

    document.getElementById('total').textContent = j.total_steps;
    document.getElementById('pace').textContent = Math.round(j.pace_per_min);

    labels = j.labels || [];
    data = j.history || [];
    drawChart();

    document.getElementById('updated').textContent = 'Aktualizacja: ' + new Date().toLocaleTimeString();
    document.getElementById('err').textContent = '';
  }catch(e){
    document.getElementById('err').textContent = 'Błąd pobierania: ' + (e.message||e);
  }
}
```

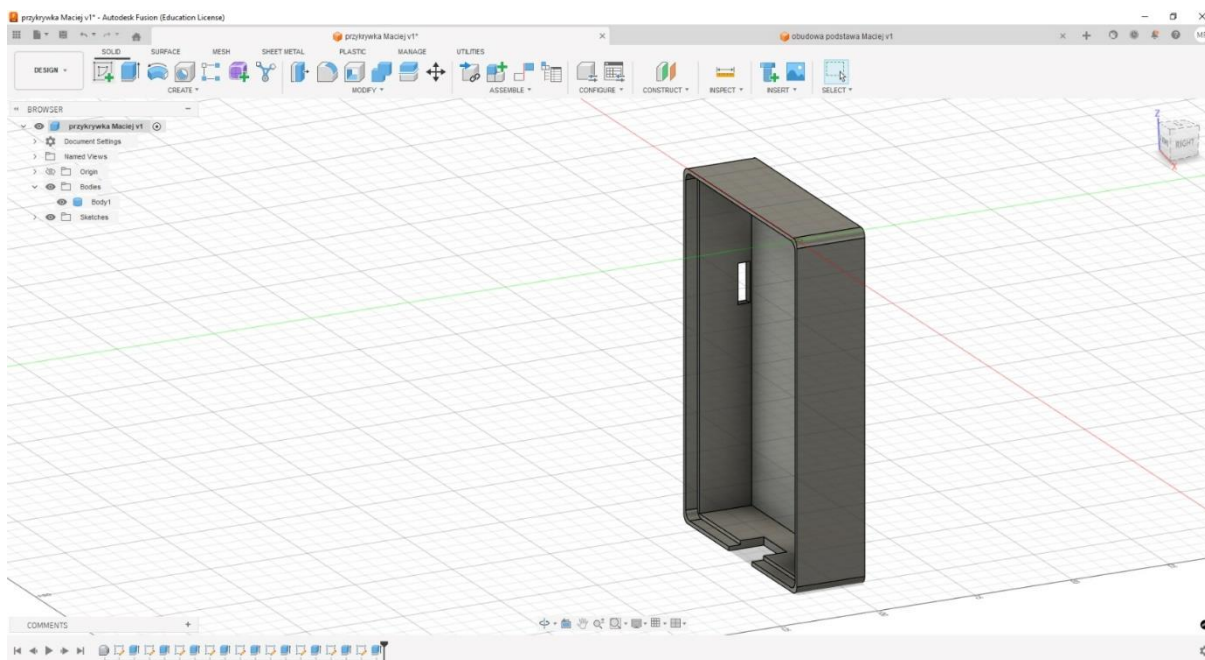
Rys. 31 Fragment skryptu JavaScript odpowiedzialny za aktualizację interfejsu użytkownika i odświeżanie wykresu



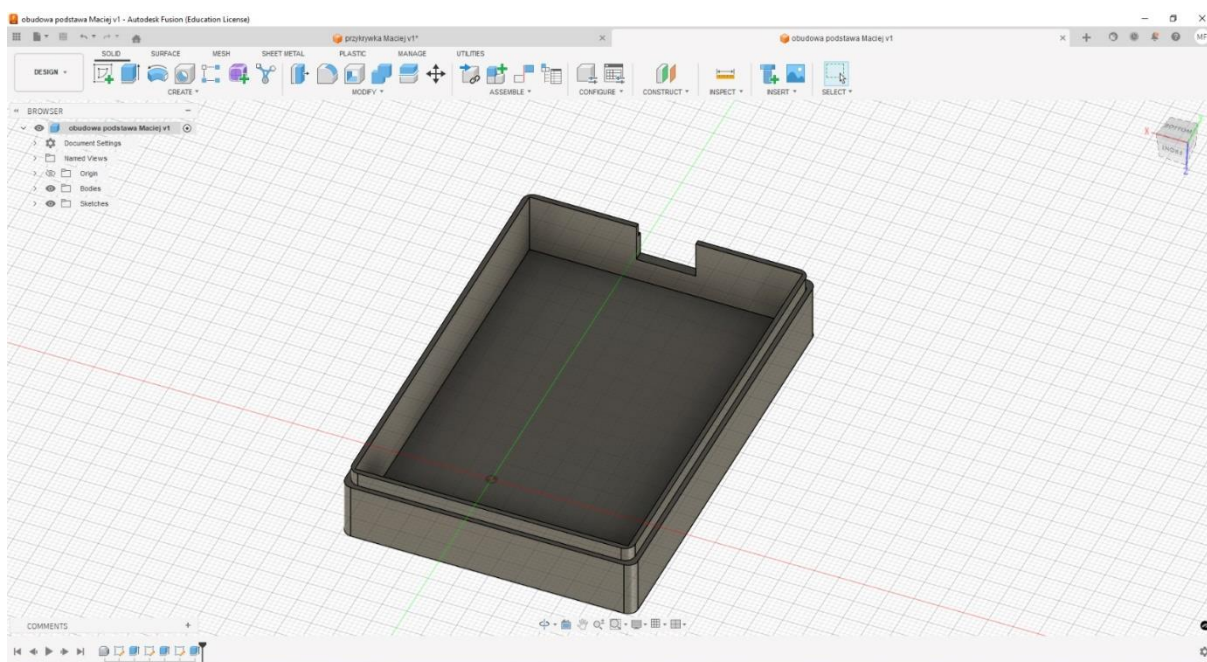
Rys. 32 Zrzut ekranu z telefonu przedstawiający interfejs graficzny użytkownika

2.6 Projekt obudowy

Prototyp odbudowy wykonałem w programie Autodesk Fusion, została ona dostosowana do wymiarów płytki stykowej i założeń o kompaktowych wymiarach projektu. Całość obudowy składa się z dwóch części – podstawy oraz pokrywki. W obu częściach składowych przewidziano miejsce na wejście kabla USB z którym połączony jest mikrokontroler. Wysokość dobrałem w taki sposób, aby w środku opakowania było wystarczająco dużo miejsca na okablowanie, umieszczenia mikrokontrolera oraz modułu MPU 6050. Podczas procesu projektowania, aby zachować precyzję w wymiarowaniu pudełka skorzystałem z suwmiarki i linijki. Podstawa obudowy ma wymiary 86,50 x 59 x 12 mm, do spodu podstawy za pomocą taśmy klejącej przytwierdzona jest płytki stykowa. Górna część obudowy ma za zadanie ochraniać komponenty znajdujące się w środku przed pyłem, deszczem i uszkodzeniami mechanicznymi. Pokrywka jest tylko nieznacznie wyższa i ma wymiary 86,5 x 59 x 17,5 mm. Po prawej stronie, gdzie będzie przytwierdzony wyświetlacz OLED został stworzony otwór idealnie dopasowany do wyprowadzeń pinów wyświetlacza. Ostateczną konstrukcję zapisałem w formacie STEP.



Rys. 33 Model pokrywy obudowy w programie Autodesk Fusion

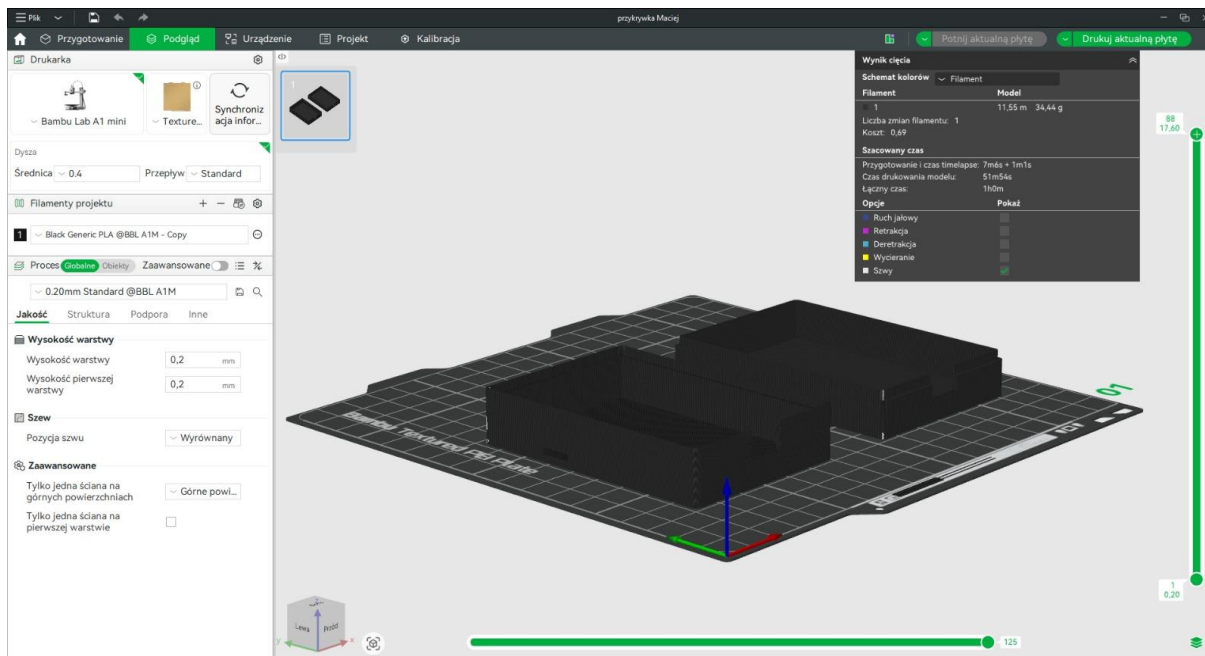


Rys. 34 Model podstawy obudowy w programie Autodesk Fusion

2.6.1 Wydruk prototypu obudowy

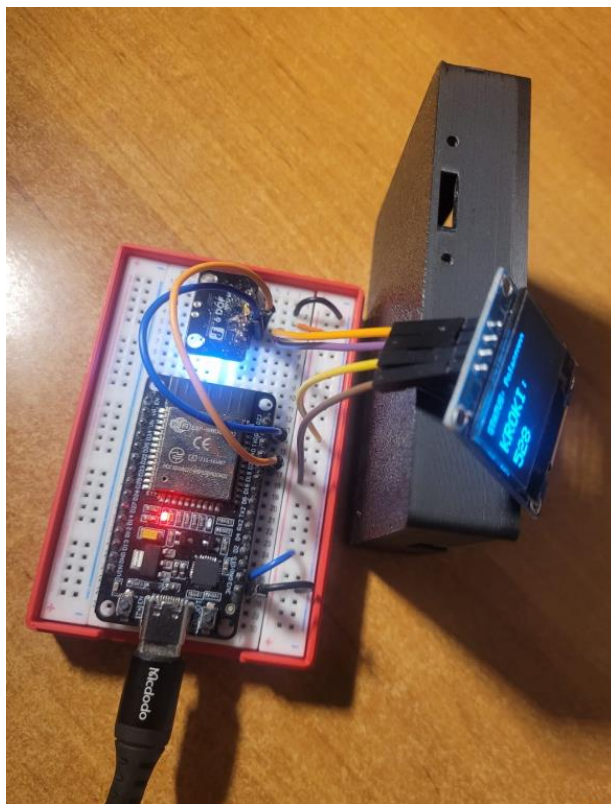
Proces wydruku rozpocząłem od importu wcześniej zapisanych plików STEP, do oprogramowania Bambu Lab Studio. Umieściłem wirtualne płytki na matrycy. Slicer podzielił je na cienkie warstwy oraz wygenerował G-code kluczowy dla pracy drukarki, gdyż zawiera on wszystkie potrzebne do wydruku informacje. Konfiguracje parametrów druku zacząłem od ustawienia grubości warstwy na 0,25 mm oraz wypełnienia na 15%. Temperaturę ustawiłem na

220°, co jest standardem przy pracy z filamentem PLA. Po zatwierdzeniu wszystkich zmian rozpoczęło się przesyłanie projektu do drukarki. Drukarka, z której skorzystałem to Bambu Lab A1 Mini wyposażoną w dyszę 0,4 mm. Wybór tego urządzenia był motywowany wysoką precyzją wydruków wymaganą do tworzenia funkcjonalnych prototypów.



Rys. 35 Wizualizacja z programu Bambu Lab Studio

Proces wydruku trwał dokładnie jedną godzinę, był bezproblemowy i przebiegał stabilnie. Do wykonania prototypu zużyto około 34 gramów filamentu Black Generic PLA, na którego wybór zdecydowałem się ze względu na świetny stosunek ceny do jakości i wytrzymałości. W efekcie powstała estetyczna i minimalistyczna obudowa w pełni realizująca założenia projektowe.



Rys. 36 Grafika przedstawia rozmieszczenie komponentów elektronicznych wewnątrz obudowy



Rys. 37 Prototyp zamknięty w obudowie i przymocowany do paska spodni

2.7 Testy poprawności działania

2.7.1 Metodyka badań

Przeprowadzone badania miały na celu zweryfikowanie poprawności działania prototypu i skuteczności w zliczaniu kroków opracowanego algorytmu w porównaniu z innymi komercyjnymi rozwiązaniami. Testy zostały przeprowadzone w warunkach rzeczywistych podczas marszu na płaskiej nawierzchni asfaltowej. Badania odbywały się na krótkich dystansach 300 kroków oraz długich, gdzie liczba kroków powinna wynosić równo 3000. Jako punkt odniesienia przyjęto manualne zliczanie kroków. Dodatkowo na obu dystansach uwzględniono dwa warianty tempa chodu: wolny, spacerowy krok oraz szybki marsz. Jako urządzenie porównawcze wybrano aplikację krokomierza na smartfonie, który znajdował się w dokładnie tej samej lokalizacji co prototyp – na wysokość pasa biodrowego.

2.7.2 Zestawienie wyników

Na odcinkach krótkodystansowych (300 kroków), przy wolnym tempie marszu, prototyp krokomierza wykazał się zaskakującą precyzją zliczając 301 kroków (błąd na poziomie 0,33%), podczas gdy aplikacja krokomierza w telefonie wskazała 309 kroków. Przy zmianie dynamiki chodu na szybsze tempo oba wykazały tendencję do nadmiarowego zliczania: prototyp zarejestrował 314 kroków co generuje błąd wynoszący 4,67%, natomiast aplikacja komercyjna 316 kroków.

Weryfikacja na długich dystansach (3000 kroków) potwierdziła stabilność prototypu. Przy wolnym tempie chodu prototyp zliczył 3011 kroków, utrzymując błąd na bardzo niskim poziomie 0,37%. W tych samych warunkach aplikacja smartfonowa naliczyła 3042 kroki. Podczas szybkiego marszu prototyp krokomierza zarejestrował 3137 kroków co skutkowało błędem wynoszącym 4,57%, podczas gdy rozwiązanie komercyjne wskazało na 3187 kroków.

2.7.3 Analiza porównawcza

Zestawienie tych danych wskazuje, że algorytm oparty na platformie ESP32 cechuje się zdecydowanie wyższą dokładnością pomiarową niż aplikacja mobilna. Różnica jest szczególnie widoczna, gdy tempo przemieszczania się ma charakter spokojnego spaceru. Wtedy prototyp krokomierza odnotowywał najwyższą precyzję a błąd na długim dystansie wynosił poniżej 0,4%. Przy szybkim tempie wzrastał do około 4,6% co spowodowane jest znacznie większą dynamiką ruchu i powstaniem drgań wtórnych, które mogły być interpretowane przez algorytm jako kroki. Warto zaznaczyć jednak, że pomimo gorszego wyniku, wciąż prezentował się lepiej niż aplikacja

krokomierza, a wynik błędu poniżej 5% jest akceptowalny. Przy porównaniu wyników na krótszych i dłuższych dystansach, można zauważyć stabilizację błędu względnego. W przypadku szybkiego marszu błąd spadł z 4,67% do 4,57%. Można zatem wnioskować, że algorytm nie kumuluje błędów przypadkowych, a różnica wynika z błędu systematycznego filtracji. We wszystkich testach opracowane urządzenie spisało się lepiej niż aplikacja mobilna co jest wynikiem wysokiej częstotliwości próbkowania i dopracowania warunków kroku algorytmu. Błędy które odnotowano zawsze wynikają z powodu za dużej ilości zliczonych kroków, co może wskazywać na to, że algorytm jest zbyt czuły.

2.7.4 Wpływ charakteru trasy na błąd pomiarowy

Ostatnim etapem testów było zbadanie wpływu geometrii trasy na pracę prototypu. Badanie miało sprawdzić jak siły odśrodkowe i rotacja tułowia wpływają na precyzję zliczania kroków. Test został przeprowadzony na dystansie 300 kroków, idąc po okręgu o promieniu około 3 metrów. Końcowe wyniki zestawiono z danymi pochodzącymi od wbudowanej aplikacji krokomierza w telefonie.

- Odczyt z prototypu ESP32: 311 kroków (błąd bezwzględny wynosi 3,67%),
- Odczyt komercyjnej aplikacji mobilnej: 309 kroków (błąd bezwzględny wynosi 3,00%).

W przypadku trasy po ciasnym okręgu na dystansie 300 kroków błąd wzrósł w porównaniu z trasą na prostej nawierzchni do 3,67%. Jest to przewidywalne zjawisko, gdyż podczas poruszania się po okręgu środek ciężkości użytkownika przemieszcza się nie tylko w pionie, ale ulega także stałemu odchyleniu bocznemu. Powoduje to powstanie dodatkowych składowych przyspieszenia, które przez algorytm mogą być interpretowane jako drgania towarzyszące krokom. Zbliżone wyniki prototypu oraz aplikacji świadczą o konkurencyjnej precyzji opracowanego urządzenia.

Podsumowanie i wnioski

W niniejszej pracy inżynierskiej skupiłem się na zaprojektowaniu prototypu elektronicznego krokomierza opartego na mikrokontrolerze ESP32 oraz czujniku inercyjnym MPU 6050. Po przeprowadzonych w terenie testach pracy można stwierdzić, że urządzenie jest w pełni sprawne i funkcjonalne.

Postawienie na ESP32 okazało się być bardzo trafne, ze względu na wbudowany moduł komunikacji bezprzewodowej oraz wysoką moc obliczeniową pozwalającą na przetworzenie danych z akcelerometru. Wykorzystanie powerbanku od firmy Hama zapewniło stabilne zasilanie i wysoką mobilność urządzenia. Zaimplementowany algorytm detekcji kroków okazał się bardzo precyzyjny, osiągając na długim dystansie błąd pomiarowy na poziomie 0,37% podczas spaceru oraz 4,57% przy szybkim tempie marszu. Wyniki te są konkurencyjne do komercyjnych propozycji krokomierzy. Stworzony interfejs użytkownika w formie strony internetowej, której serwer znajduje się bezpośrednio wewnątrz ESP32 pozwala na efektywny i wygodny podgląd wyników. Wyświetlacz OLED również świetnie się sprawdził zapewniając bieżący podgląd kroków.

Opracowany model, mimo osiągnięcia założonych celów posiada jeszcze potencjał do dalszego rozwoju i budowy bardziej zaawansowanego systemu. Jedynym z kierunków rozwoju projektu jest optymalizacja zużycia energii poprzez wprowadzenie trybu uśpienia mikrokontrolera w momentach braku wykrycia ruchu, co wydłużyłoby czas pracy baterii. Interesujące byłoby również wykorzystanie wbudowanego w ESP32 modułu Bluetooth Low Energy i zbudowanie dedykowanej mobilnej aplikacji. Aby skierować projekt w stronę bardziej profesjonalnego asystenta aktywności fizycznej należałoby skupić się na rozwoju i modyfikacji oprogramowania, co pozwoliłoby na osiągnięcie wyników wskazujących mniejszy błąd pomiarowy, precyzyjniejsze eliminowanie drgań oraz automatyczne rozpoznawanie aktywności ruchowej. Integracja urządzenia z chmurą lub zewnętrzną bazą danych umożliwiłaby długoterminową archiwizację danych pomocną przy śledzeniu trendów aktywności fizycznej użytkownika.

Bibliografia

- [1]Kavanagh J. J., Menz H. B.: Accelerometry: A technique for quantifying movement patterns during walking. Gait & Posture. Elsevier B.V. July 2008. Pages 1-15
- [2]https://www.neonet.pl/blog/smartband-mi-band-5-jak-wlaczyc-te-opaske.html?srsId=AfmBOoqpwaJdaF5176U7w_dFfxe9Rqxhnp-8fvELMio7zSs8d141WF9I
- [3]<https://botland.com.pl/produkty-wycofane/16022-smartband-xiaomi-mi-band-4-czarny-inteligentna-opaska-6934177710377.html>
- [4]Działanie akcelerometru:
<https://www.tme.eu/pl/news/library-articles/page/22568/Akcelerometr-jak-dziala-i-do-czego-sluzyl>; stan na dzień: 19.11.2025
- [5]<https://mikrokontroler.pl/2012/07/05/inemo-stm32f103-i-mems-y/>
- [6]Prawidłowe wzorce chodu:
<https://bieganie.pl/zdrowie/prawidlowe-wzorce-chodu-ktore-pomoga-ci-zdrowo-biegac/>
- [7]Biomechanika chodu:
https://teach.ibv.org/wp-content/uploads/sites/18/2021/07/BG_A_Didactic_Unit_PL.pdf
- [8]<https://botland.com.pl/blog/rozmowy-miedzy-maszynami-standardy-komunikacji-w-ekosystemie-iot/#Komunikacja-w-swiecie-Internetu-Rzeczy>
- [9]<https://botland.com.pl/blog/magistrala-i2c-co-to-jest-i-do-czego-sluzyl>
- [10]<https://ep.com.pl/rynek/elektronika-w-praktyce/16152-komunikacja-bezprzewodowa-w-iot>
- [11]<https://www.hotgear.pl/sieci-wi-fi-w-praktyce-czym-roznia-standardy-802-11b-a-g-n-ac-ax-czyli-wi-fi-1-2-3-4-5-6/>
- [12]https://sklep.msalamon.pl/produkt/esp32-devkitc-esp-wroom-32u/?gad_source=1&gad_campaignid=21700502614&gbraid=0AAAAABYhW9DujwmLrvMAkoJ3IZIsuNSrn&gclid=Cj0KCQiAx8PKBhD1ARIsAKsmGb74QPjy5sYiJnKfRNNEb6WNueTdk32x_p0qhsO-YiLb2a2TTWuC8oaAs61EALw_wcB ;stan na dzień 28.11.2025:
- [13]<https://www.hns-berks.co.uk/blogs/24ghz-and-5ghz-explained>
- [14]<https://www.morele.net/wiadomosc/jakie-pasmo-wi-fi-wybrac-2-4ghz-czy-5ghz/17027/>
- [15]<https://tplinkforum.pl/t/zakres-czestotliwosci-dla-sieci-wi-fi-na-pasmie-2-4-ghz-i-5-ghz/8519>

- [16]<https://botland.com.pl/esp32/8893-esp32-wifi-bt-42-platforma-z-modulem-esp-wroom-32-zgodny-z-esp32-devkit-5904422337438.html> ;stan na dzień 28.11.2025
- [17]<https://i0.wp.com/randomnerdtutorials.com/wp-content/uploads/2018/08/ESP32-DOIT-DEVKIT-V1-Board-Pinout-30-GPIOs-Copy.png>
- [18]<https://embedded-systems-design.github.io/overview-of-the-esp32-devkit-doit-v1/SchematicsforESP32.pdf> ;stan na dzień 28.11.2025
- [19]Baza danych komponentów elektronicznych, NCP1117:
<https://partsbox.com/pl/ecdb/ncp1117st33t3g-ldo-regulator-onsemi.html>
- [20]https://elektroweb.pl/pl/zyroskopy/45-zyroskop-mpu-6050-gy-521-akcelerometr-3-osiowy-do-rc-arduino-rpi.html?gad_source=1&gad_campaignid=22555279180&gbraid=0AAAAADsUy2bRg8sv4tvCJMR1HZR9v_EIj&gclid=CjwKCAiAu67KBhAkEiwAY0jAlW232iPezp2R98haJMPzLX6hIAzib0NI_nCBJxaOhWhe0IxZD-iJKhoCrOMQAvD_BwE ;stan na dzień 28.11.2025
- [21]<https://www.electronicwings.com/sensors-modules/mpu6050-gyroscope-accelerometer-temperature-sensor-module>
- [22]<https://forum.arduino.cc/t/mpu-6050-schematic-question/402839>
- [23]<https://botland.com.pl/wyswietlacze-oled/8867-wyswietlacz-oled-niebieski-graficzny-13-128x64px-i2c-v2-niebieskie-znaki-sh1106-5903351241182.html> ;stan na dzień 29.11.2025
- [24]<https://mikrokontroler.pl/2016/09/23/mbed-stm32-ssd1306-sterowanie-dwukolorowym-oled-1/>
- [25]<https://forbot.pl/blog/jak-dziala-plytka-stykowa-zdjecia-budowa-przyklady-id21978>
- [26]<https://manuals.plus/pl/hama/00187292-supreme-20hd-20000mah-power-pack-manual>

Spis rysunków

Rys. 1 Inteligentna opaska Xiaomi Mi Band 4 [2]	6
Rys. 2 Grafika przedstawia działanie żyroskopu w technologii MEMS [5]	8
Rys. 3 Zdarzenia oraz fazy w cyklu chodu człowieka [6]	9
Rys. 4 Płaszczyzny ciała człowieka [7]	10
Rys. 5 Ewolucja standardów Wi-Fi IEEE 802.11 [11].....	12
Rys. 6 Obrazowe przedstawienie różnicy w częstotliwości [13]	12
Rys. 7 Porównanie kanałów transmisyjnych [15].....	13
Rys. 8 Moduł ESP32 Dev Kit V1[16]	15
Rys. 9 Schemat wyprowadzeń pinów na płytce ESP32 Dev Kit V1 [17]	16
Rys. 10 Schemat sekcji zasilania ESP32 Dev Kit V1 [18]	16
Rys. 11 Moduł pomiarowy MPU 6050 [20].....	18
Rys. 12 Grafika przedstawia trzyosiowy żyroskop [21]	19
Rys. 13 Orientacja osi pomiarowych i kątów obrotu modułu MPU 6050 [21].....	20
Rys. 14 Schemat modułu MPU 6050 [22]	21
Rys. 15 Moduł wyświetlacza OLED [23]	22
Rys. 16 Schemat blokowy sterownika SSD1306 [24].....	23
Rys. 17 Płytki stykowa [25]	24
Rys. 18 Powerbank Hama Supreme 20HD [26].....	24
Rys. 19 Zrzut ekranu z programu KiCad przedstawiający schemat połączeń komponentów	26
Rys. 20 Schemat blokowy algorytmu działania oprogramowania	27
Rys. 21 Implementacja bibliotek zrzut ekranu z środowiska Arduino IDE	28
Rys. 22 Zrzut ekranu z programu Arduino Uno pokazujący wzór przyspieszenia całkowitego, obliczanego ze składowych a_x , a_y , a_z akcelerometru	30
Rys. 23 Zrzut ekranu przedstawiający filtr.....	30
Rys. 24 Zrzut ekranu przedstawiający warunki do detekcji kroku.....	30
Rys. 25 Zrzut ekranu z programu Arduino Uno który pokazuje inkrementację licznika, jeśli spełnione są kryteria kroku.....	31
Rys. 26 Konfiguracja danych uwierzytelniających sieci Wi-Fi (SSID i hasło) oraz definicja parametrów wyświetlacza OLED.....	31
Rys. 27 Zrzut ekranu ze środowiska Arduino Uno przedstawiający proces łączenia się siecią Wi-Fi oraz wyświetlanie adresu IP serwera po nawiązaniu połączenia	32
Rys. 28 Proces inicjalizacji wyświetlacza OLED.....	32
Rys. 29 Fragment kodu odpowiadający za archiwizację danych.....	32
Rys. 30 Fragment kodu odpowiedzialny za generowanie i wysyłanie danych w formacie JSON.....	33
Rys. 31 Fragment skryptu JavaScript odpowiedzialny za aktualizację interfejsu użytkownika i odświeżanie wykresu.....	33
Rys. 32 Zrzut ekranu z telefonu przedstawiający interfejs graficzny użytkownika	34

Rys. 33 Model pokrywki obudowy w programie Autodesk Fusion.....	35
Rys. 34 Model podstawy obudowy w programie Autodesk Fusion.....	35
Rys. 35 Wizualizacja z programu Bambu Lab Studio	36
Rys. 36 Grafika przedstawia rozmieszczenie komponentów elektronicznych wewnątrz obudowy.....	37
Rys. 37 Prototyp zamknięty w obudowie i przymocowany do paska spodni	37